

MISI 2 DESAIN FISIKAL DAN DEVELOPMENT

Data Warehouse



Disusun oleh kelompok 4:

1. Adil Aulia Rahma Nurhidayah (122450058)
2. Rosalia Siregar (123450036)
3. Muhammad Hanif Dzaky Arifin (123450064)
4. Haikall Fransisko Simbolon (123450106)

**PROGRAM STUDI SAINS DATA
FAKULTAS SAINS
INSTITUT TEKNOLOGI SUMATERA
2025**

A. Deskripsi Misi

Misi 2 berfokus pada pembangunan Data Warehouse Kemahasiswaan ITERA berdasarkan desain dimensional yang telah dibuat pada Misi 1. Pada tahap ini dilakukan implementasi fisik database menggunakan SQL Server, pembuatan tabel dimensi dan fakta, indexing serta partitioning untuk mendukung performa query analitis. Selain itu, dirancang dan diimplementasikan proses ETL untuk melakukan ekstraksi data dari sistem operasional seperti SIAKAD dan sistem pelaporan Ormawa, kemudian mentransformasikan data ke dalam format yang konsisten dan terintegrasi sebelum dimuat ke dalam Data Warehouse.

Tahap ini juga mencakup verifikasi kualitas data dan pengujian performa sebagai persiapan deploy ke lingkungan produksi.

B. Tujuan Misi

Misi 2 bertujuan untuk:

1. Mengimplementasikan desain logikal Star Schema Kemahasiswaan ke dalam basis data SQL Server.
2. Mengembangkan proses ETL yang mampu memuat data mahasiswa, kegiatan, organisasi, dan partisipasi ke Data Warehouse.
3. Melakukan optimasi performa melalui indexing, partitioning, dan teknik tuning query.
4. Menjamin data quality dengan validasi integritas data, konsistensi referensial, dan pendeteksian duplikasi.
5. Menyiapkan infrastruktur Data Warehouse untuk mendukung analisis dan pelaporan aktivitas kemahasiswaan.

C. Deliverables

1. SQL DDL Scripts (CREATE TABLE, INDEX, PARTITION)
2. ETL Package (SSIS atau scripts)
3. Data Quality Report
4. Performance Optimization Documentation
5. Testing Documentation
6. Updated GitHub Repository

Step 1: Physical Database Design

Data Warehouse Kemahasiswaan dibangun menggunakan pendekatan dimensional modeling dengan arsitektur Star Schema, terdiri dari:

1. Database Setup

```
CREATE DATABASE DM_Kemahasiswaan_DW
ON PRIMARY
(
    NAME = N'DM_Kemahasiswaan_DW_Data',
    FILENAME = N'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\DM_Kemahasiswaan_DW_Data
.mdf', -- Ganti dengan jalur default server Anda
    SIZE = 1GB,
```

```

MAXSIZE = UNLIMITED ,
FILEGROWTH = 256MB
)
LOG ON
(
    NAME = N'DM_Kemahasiswaan_DW_Log',
    FILENAME = N'C:\Program Files\Microsoft SQL
Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\DM_Kemahasiswaan_DW_Log.
ldf', -- Ganti dengan jalur default server Anda
    SIZE = 256MB,
    MAXSIZE = 2GB,
    FILEGROWTH = 64MB
);
GO

```

```

1 CREATE DATABASE DM_Kemahasiswaan_DW
2 ON PRIMARY
3 (
4     NAME = N'DM_Kemahasiswaan_DW_Data',
5     FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\DM_Kemahasiswaan_DW_Data.mdf',
6     SIZE = 1GB,
7     MAXSIZE = UNLIMITED ,
8     FILEGROWTH = 256MB
9 )
10 LOG ON
11 (
12     NAME = N'DM_Kemahasiswaan_DW_Log',
13     FILENAME = N'C:\Program Files\Microsoft SQL Server\MSSQL16.SQLEXPRESS\MSSQL\DATA\DM_Kemahasiswaan_DW_Log.ldf',
14     SIZE = 256MB,
15     MAXSIZE = 2GB,
16     FILEGROWTH = 64MB
17 );
18 GO

```

Database DM_Kemahasiswaan_DW dibuat sebagai basis Data Warehouse kemahasiswaan dengan konfigurasi penyimpanan yang dirancang untuk mendukung pengolahan data berskala besar dan bersifat historis. Database ini memiliki dua komponen utama, yaitu file data (.mdf) dan file log (.ldf). File data diatur dengan ukuran awal 1GB, dapat berkembang tanpa batas (unlimited), serta memiliki mekanisme pertumbuhan otomatis sebesar 256MB untuk menjaga performa dan meminimalkan fragmentasi. Sementara itu, file log memiliki ukuran awal 256MB, dibatasi hingga maksimum 2GB, dan bertambah secara otomatis sebesar 64MB untuk mengontrol penggunaan ruang disk dan menjaga stabilitas transaksi. Kedua file ditempatkan pada direktori default SQL Server Express, dengan catatan bahwa jalur penyimpanan perlu disesuaikan dengan konfigurasi server yang digunakan.

2. Create Dimension Tables

```

-- 02_Create_Dimensions.sql

-- Dimension Table: Dim_Tanggal (Disesuaikan dari contoh)
CREATE TABLE dbo.Dim_Tanggal (
    Tanggal_SK INT PRIMARY KEY NOT NULL,

```

```

FullDate DATE NOT NULL,
DayNumberOfWeek TINYINT NOT NULL,
DayName VARCHAR(10) NOT NULL,
MonthName VARCHAR(10) NOT NULL,
MonthNumber TINYINT NOT NULL,
Quarter TINYINT NOT NULL,
QuarterName VARCHAR(6) NOT NULL,
Tahun SMALLINT NOT NULL,
IsWeekend BIT NOT NULL,
Semester VARCHAR(10) NULL, -- Ganjil/Genap
CONSTRAINT CK_Dim_Tanggal_Quarter CHECK (Quarter BETWEEN 1
AND 4),
CONSTRAINT CK_Dim_Tanggal_Month CHECK (MonthNumber BETWEEN
1 AND 12)
);
GO

```

```

-- Dimension Table: Dim_Mahasiswa (Student - SCD Type 2)
CREATE TABLE dbo.Dim_Mahasiswa (
Mahasiswa_SK INT IDENTITY(1,1) PRIMARY KEY NOT NULL,
NIM VARCHAR(20) UNIQUE NOT NULL, -- Natural Key
Nama_Mahasiswa VARCHAR(100) NOT NULL,
Fakultas VARCHAR(100) NOT NULL,
Program_Studi VARCHAR(100) NOT NULL,
Tahun_Masuk SMALLINT NOT NULL,
Status_Mahasiswa VARCHAR(20) NOT NULL, -- Active, Cuti, Drop-out, etc.
-- SCD Type 2 attributes
EffectiveDate DATE NOT NULL DEFAULT GETDATE(),
ExpiryDate DATE NULL,
IsCurrent BIT NOT NULL DEFAULT 1,
CreatedDate DATETIME DEFAULT GETDATE()
);
GO

```

```

-- Dimension Table: Dim_Kegiatan
CREATE TABLE dbo.Dim_Kegiatan (
Kegiatan_SK INT IDENTITY(1,1) PRIMARY KEY NOT NULL,
ID_Kegiatan_NK VARCHAR(50) UNIQUE NOT NULL, -- Natural Key dari
OLTP
Nama_Kegiatan VARCHAR(255) NOT NULL,
Jenis_Kegiatan VARCHAR(50) NOT NULL, -- Seminar, Lomba, Baksos
Skala_Kegiatan VARCHAR(20) NOT NULL, -- Internal, Nasional, Internasional
Status_Kegiatan VARCHAR(20) NOT NULL -- Terlaksana, Batal
);
GO

```

```

-- Dimension Table: Dim_Organisasi
CREATE TABLE dbo.Dim_Organisasi (
Organisasi_SK INT IDENTITY(1,1) PRIMARY KEY NOT NULL,

```

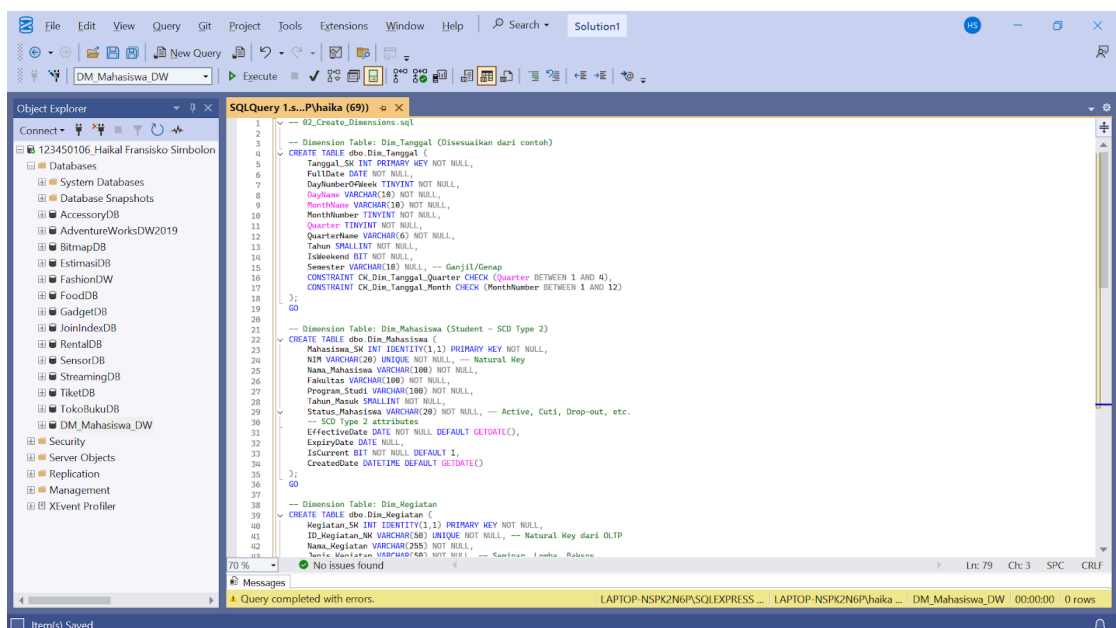
```

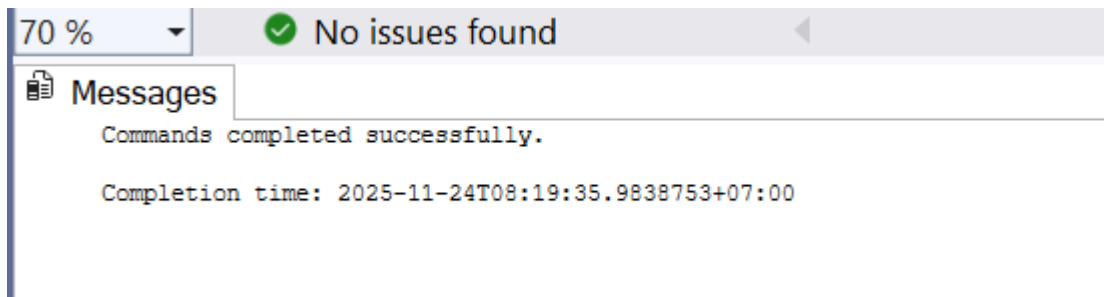
ID_Organisasi_NK VARCHAR(50) UNIQUE NOT NULL,
Nama_Organisasi VARCHAR(100) NOT NULL,
Jenis_Organisasi VARCHAR(20) NOT NULL, -- UKM atau Ormawa
Status_Organisasi VARCHAR(10) NOT NULL, -- Aktif/Non-aktif
EffectiveDate DATE NOT NULL DEFAULT GETDATE(),
IsCurrent BIT NOT NULL DEFAULT 1 -- SCD Type 2 untuk status
);
GO

-- Dimension Table: Dim_Prestasi
CREATE TABLE dbo.Dim_Prestasi (
    Prestasi_SK INT IDENTITY(1,1) PRIMARY KEY NOT NULL,
    ID_Prestasi_NK VARCHAR(50) UNIQUE NOT NULL,
    Nama_Penghargaan VARCHAR(255) NOT NULL,
    Level_Prestasi VARCHAR(50) NOT NULL, -- Regional, Nasional, Internasional
    Kategori_Prestasi VARCHAR(50) NOT NULL -- Akademik atau Non-akademik
);
GO

-- Dimension Table: Dim_Beasiswa
CREATE TABLE dbo.Dim_Beasiswa (
    Beasiswa_SK INT IDENTITY(1,1) PRIMARY KEY NOT NULL,
    ID_Beasiswa_NK VARCHAR(50) UNIQUE NOT NULL,
    Nama_Beasiswa VARCHAR(100) NOT NULL,
    Sumber_Dana VARCHAR(50) NOT NULL, -- Internal/Eksternal
    Periode_Berlaku VARCHAR(20) NOT NULL
);
GO

```





Mendefinisikan enam tabel dimensi pada Data Warehouse kemahasiswaan yang masing-masing berfungsi sebagai data referensi analitis. Dim_Tanggal merupakan dimensi waktu dengan primary key Tanggal_SK yang merepresentasikan tanggal secara lengkap, mencakup hari, bulan, kuartal, tahun, serta informasi semester (Ganjil/Genap), dan dilengkapi constraint untuk menjaga validitas nilai bulan dan kuartal. Dim_Mahasiswa menerapkan konsep Slowly Changing Dimension (SCD) Type 2 untuk melacak perubahan historis data mahasiswa seperti status, fakultas, dan program studi, dengan atribut EffectiveDate, ExpiryDate, dan IsCurrent, serta menggunakan NIM sebagai natural key. Dim_Kegiatan menyimpan informasi kegiatan kemahasiswaan berdasarkan jenis, skala, dan status pelaksanaan.

Dim_Organisasi juga menggunakan pendekatan SCD Type 2 untuk memantau perubahan status organisasi (Aktif/Non-aktif) pada jenis UKM dan Ormawa. Dim_Prestasi mencatat data penghargaan mahasiswa berdasarkan level dan kategori prestasi, sedangkan Dim_Beasiswa menyimpan informasi beasiswa meliputi nama, sumber dana, dan periode berlaku. Seluruh tabel dimensi menggunakan surrogate key (_SK) sebagai primary key, dengan Dim_Tanggal menggunakan key berbasis tanggal dan tabel lainnya menggunakan IDENTITY, serta natural key (_NK) untuk menjaga keterkaitan dengan sistem OLTP sumber.

3. Create Fact Tables

```
-- 03_Create_Facts.sql

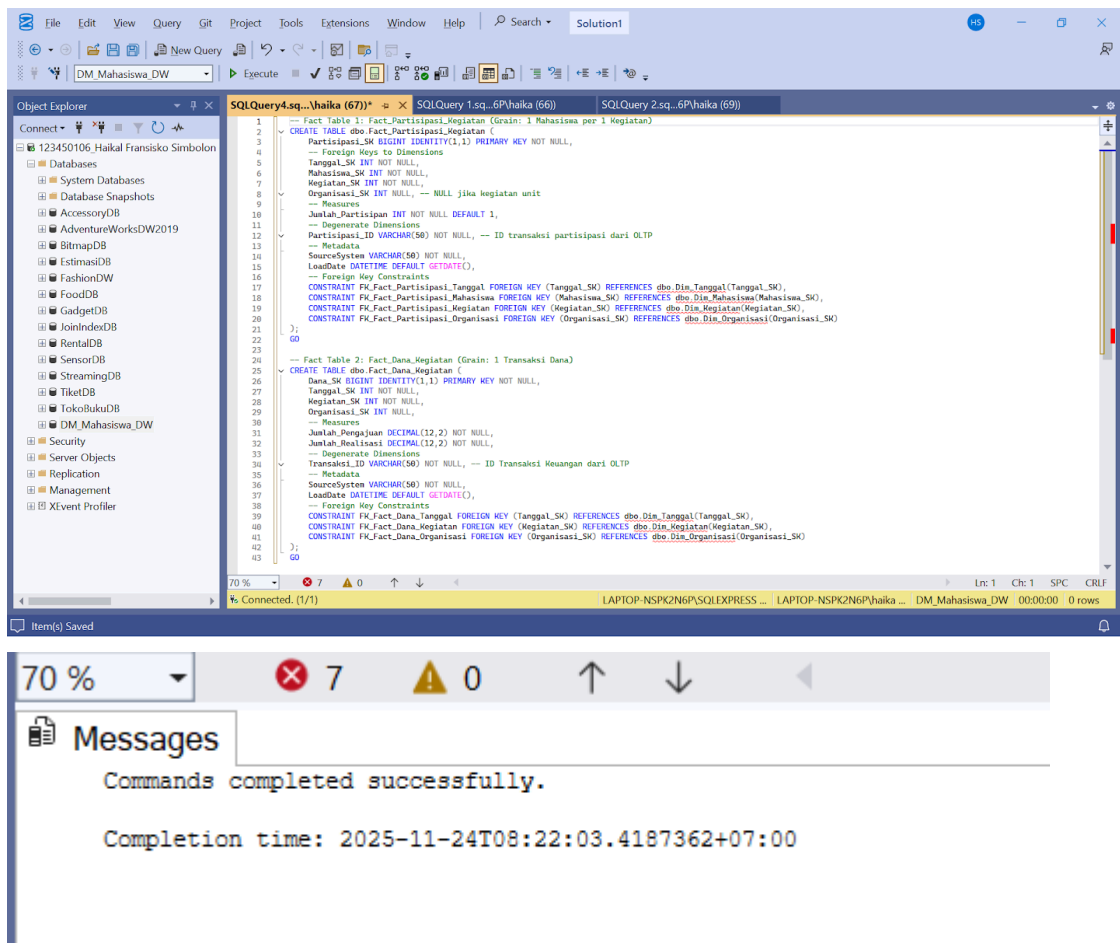
-- Fact Table 1: Fact_Partisipasi_Kegiatan (Grain: 1 Mahasiswa per 1 Kegiatan)
CREATE TABLE dbo.Fact_Partisipasi_Kegiatan (
    Partisipasi_SK BIGINT IDENTITY(1,1) PRIMARY KEY NOT NULL,
    -- Foreign Keys to Dimensions
    Tanggal_SK INT NOT NULL,
    Mahasiswa_SK INT NOT NULL,
    Kegiatan_SK INT NOT NULL,
    Organisasi_SK INT NULL, -- NULL jika kegiatan unit
    -- Measures
    Jumlah_Partisipan INT NOT NULL DEFAULT 1,
    -- Degenerate Dimensions
    Partisipasi_ID VARCHAR(50) NOT NULL, -- ID transaksi partisipasi dari OLTP
```

```

-- Metadata
SourceSystem VARCHAR(50) NOT NULL,
LoadDate DATETIME DEFAULT GETDATE(),
-- Foreign Key Constraints
CONSTRAINT FK_Fact_Partisipasi_Tanggal FOREIGN KEY (Tanggal_SK)
REFERENCES dbo.Dim_Tanggal(Tanggal_SK),
CONSTRAINT FK_Fact_Partisipasi_Mahasiswa FOREIGN KEY
(Mahasiswa_SK) REFERENCES dbo.Dim_Mahasiswa(Mahasiswa_SK),
CONSTRAINT FK_Fact_Partisipasi_Kegiatan FOREIGN KEY (Kegiatan_SK)
REFERENCES dbo.Dim_Kegiatan(Kegiatan_SK),
CONSTRAINT FK_Fact_Partisipasi_Organisasi FOREIGN KEY
(Organisasi_SK) REFERENCES dbo.Dim_Organisasi(Organisasi_SK)
);
GO

-- Fact Table 2: Fact_Dana_Kegiatan (Grain: 1 Transaksi Dana)
CREATE TABLE dbo.Fact_Dana_Kegiatan (
Dana_SK BIGINT IDENTITY(1,1) PRIMARY KEY NOT NULL,
Tanggal_SK INT NOT NULL,
Kegiatan_SK INT NOT NULL,
Organisasi_SK INT NOT NULL,
-- Measures
Jumlah_Pengajuan DECIMAL(12,2) NOT NULL,
Jumlah_Realisasi DECIMAL(12,2) NOT NULL,
-- Degenerate Dimensions
Transaksi_ID VARCHAR(50) NOT NULL, -- ID Transaksi Keuangan dari OLTP
-- Metadata
SourceSystem VARCHAR(50) NOT NULL,
LoadDate DATETIME DEFAULT GETDATE(),
-- Foreign Key Constraints
CONSTRAINT FK_Fact_Dana_Tanggal FOREIGN KEY (Tanggal_SK)
REFERENCES dbo.Dim_Tanggal(Tanggal_SK),
CONSTRAINT FK_Fact_Dana_Kegiatan FOREIGN KEY (Kegiatan_SK)
REFERENCES dbo.Dim_Kegiatan(Kegiatan_SK),
CONSTRAINT FK_Fact_Dana_Organisasi FOREIGN KEY (Organisasi_SK)
REFERENCES dbo.Dim_Organisasi(Organisasi_SK)
);
GO

```



Membuat dua tabel fakta yang menyimpan data transaksi atau kejadian dalam Data Warehouse kemahasiswaan. Fact_Partisipasi_Kegiatan merupakan tabel fakta pertama dengan grain (tingkat detail) satu record untuk setiap mahasiswa yang berpartisipasi dalam satu kegiatan. Tabel ini memiliki foreign key yang menghubungkan ke empat dimensi yaitu Dim_Tanggal, Dim_Mahasiswa, Dim_Kegiatan, dan Dim_Organisasi (bersifat nullable untuk kegiatan non-organisasi). Measure atau metrik utamanya adalah Jumlah_Partisipan yang secara default bernilai 1, serta dilengkapi dengan degenerate dimension berupa Partisipasi_ID yang merupakan ID transaksi dari sistem OLTP. Tabel ini juga menyimpan metadata seperti SourceSystem untuk tracking sumber data dan LoadDate untuk mencatat waktu pemuatan data ke warehouse.

Fact_Dana_Kegiatan adalah tabel fakta kedua dengan grain satu record per transaksi dana kegiatan. Tabel ini berelasi dengan tiga dimensi melalui foreign key yaitu Dim_Tanggal, Dim_Kegiatan, dan Dim_Organisasi (nullable). Measures yang disimpan adalah Jumlah_Pengajuan dan Jumlah_Realisasi dengan tipe data DECIMAL untuk menyimpan nilai dana dalam format moneter dengan presisi hingga 2 angka desimal. Sama seperti tabel fakta sebelumnya, tabel ini juga memiliki degenerate dimension Transaksi_ID sebagai referensi ke ID transaksi keuangan di sistem OLTP, serta metadata SourceSystem dan LoadDate. Kedua tabel fakta menggunakan BIGINT sebagai tipe data primary key (dengan suffix _SK) yang

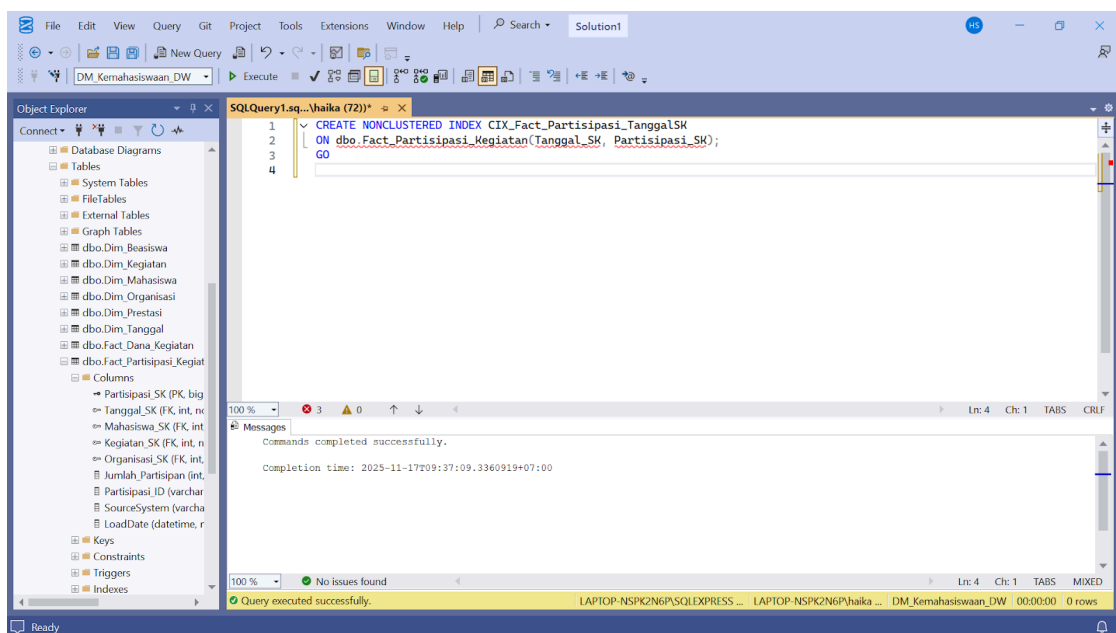
di-generate otomatis menggunakan IDENTITY untuk mengantisipasi volume data yang sangat besar, dan semua foreign key constraints didefinisikan secara eksplisit untuk menjaga integritas referensial data.

Step 2: Indexing Strategy

Tujuan : Mengoptimalkan performa query analitis pada tabel fakta dan dimensi.

1. Clustered Index on Fact Table

```
CREATE CLUSTERED INDEX CIX_Fact_Partisipasi_TanggalSK  
ON dbo.Fact_Partisipasi_Kegiatan(Tanggal_SK, Partisipasi_SK);  
GO
```



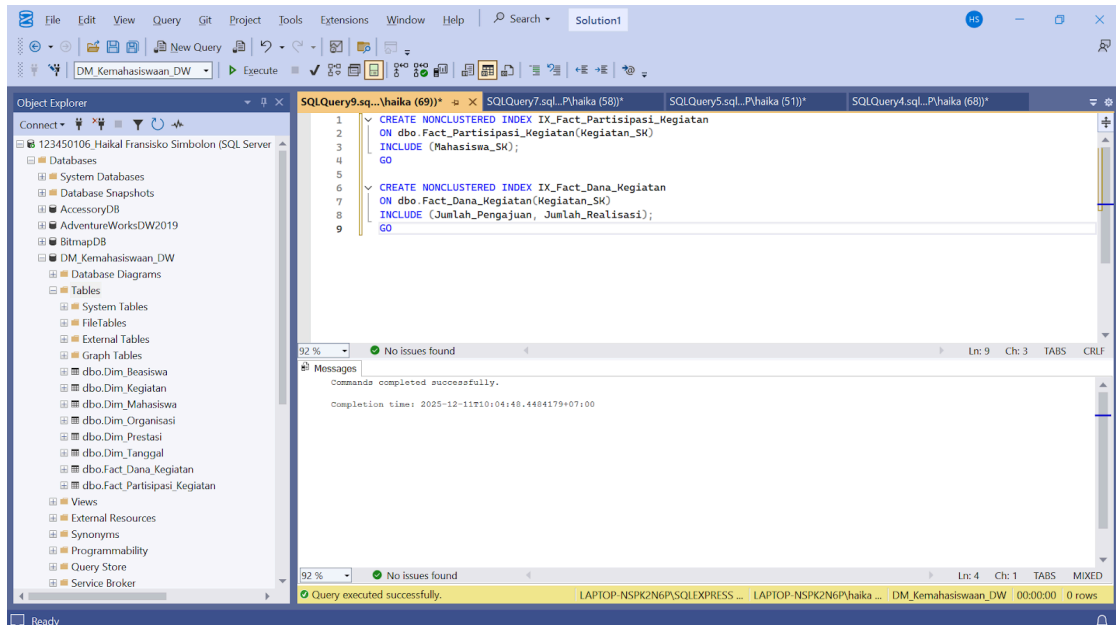
Membuat clustered index pada tabel Fact_Partisipasi_Kegiatan yang mengurutkan data secara fisik berdasarkan Tanggal_SK terlebih dahulu, lalu Partisipasi_SK yang mempercepat query melakukan filtering berdasarkan tanggal (seperti mencari data per bulan atau semester) karena data sudah tersusun berurutan di storage, yang sangat penting untuk performa analitik Data Warehouse yang sering melakukan query berdasarkan periode waktu.

2. Non-Clustered Indexes

```
-- Index untuk foreign keys (join optimization)  
CREATE NONCLUSTERED INDEX IX_Fact_Partisipasi_Kegiatan  
ON dbo.Fact_Partisipasi_Kegiatan(Kegiatan_SK)  
INCLUDE (Mahasiswa_SK);  
GO
```

```
CREATE NONCLUSTERED INDEX IX_Fact_Dana_Kegiatan
```

```
ON dbo.Fact_Dana_Kegiatan(Kegiatan_SK)
INCLUDE (Jumlah_Pengajuan, Jumlah_Realisasi);
GO
```

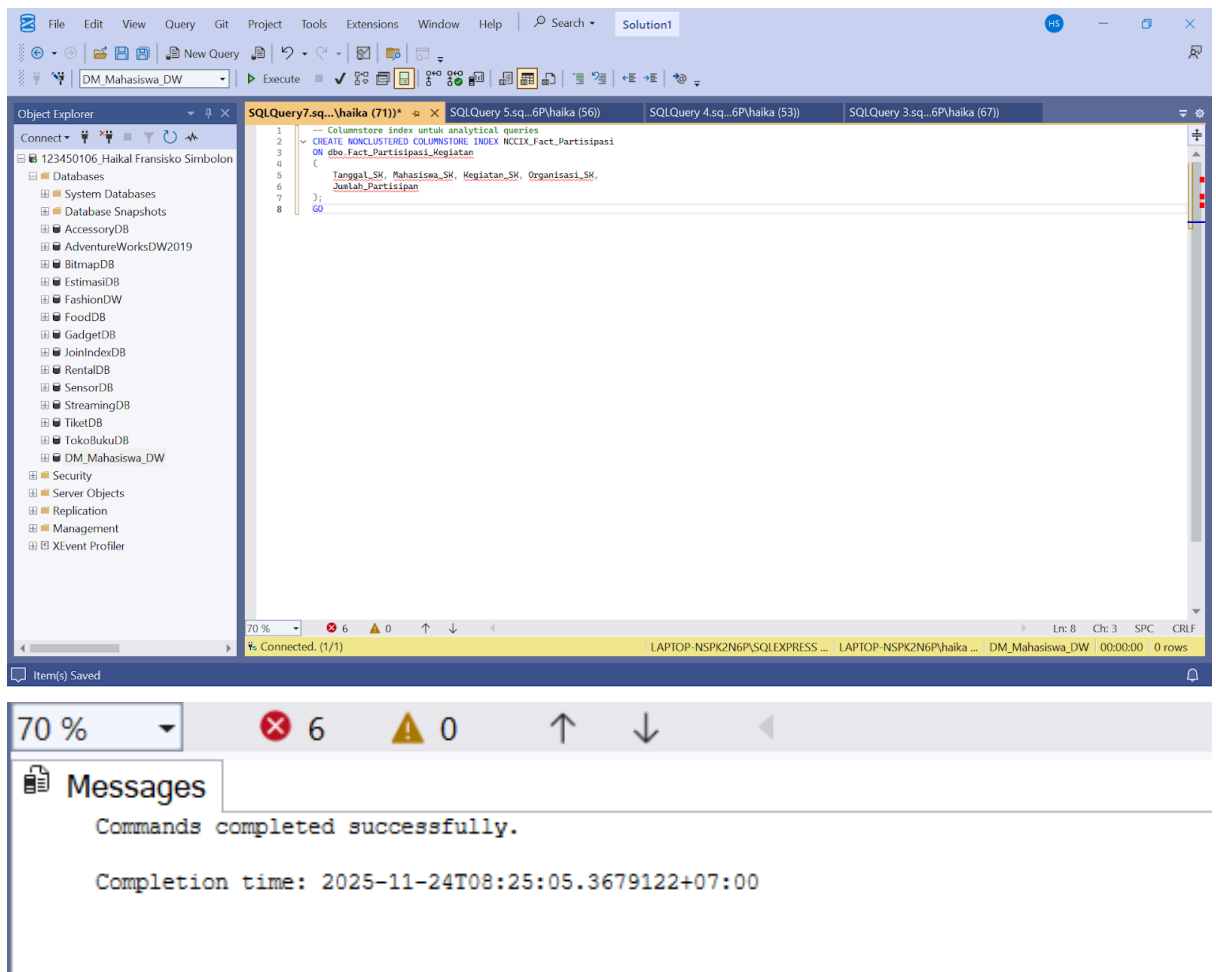


Membuat dua nonclustered index untuk mengoptimalkan performa operasi join dan query analitik pada tabel fakta. Index IX_Fact_Participasi_Kegiatan dibuat pada tabel Fact_Participasi_Kegiatan dengan kolom kunci Kegiatan_SK dan menyertakan kolom Mahasiswa_SK dalam INCLUDE clause, sehingga query yang menganalisis partisipasi mahasiswa berdasarkan kegiatan dapat dieksekusi lebih efisien tanpa harus mengakses tabel utama.

Selanjutnya, index IX_Fact_Dana_Kegiatan dibuat pada tabel Fact_Dana_Kegiatan dengan kolom kunci Kegiatan_SK serta menyertakan kolom Jumlah_Pengajuan dan Jumlah_Realisasi sebagai measure. Pendekatan ini menerapkan teknik covering index yang memungkinkan query analisis dana per kegiatan langsung mengambil seluruh data yang dibutuhkan dari index, sehingga mengurangi operasi I/O ke storage dan meningkatkan performa query analitik pada Data Warehouse.

3. Columnstore Index (for large fact tables)

```
-- Columnstore index untuk analytical queries
CREATE NONCLUSTERED COLUMNSTORE INDEX NCCIX_Fact_Participasi
ON dbo.Fact_Participasi_Kegiatan
(
    Tanggal_SK, Mahasiswa_SK, Kegiatan_SK, Organisasi_SK,
    Jumlah_Partisipan
);
GO
```



Membuat nonclustered columnstore index pada tabel Fact_Partisipasi_Kegiatan yang menyimpan data dalam format kolom (column-oriented) untuk kolom Tanggal_SK, Mahasiswa_SK, Kegiatan_SK, Organisasi_SK, dan Jumlah_Partisipan. Columnstore index sangat optimal untuk query analitik seperti agregasi (SUM, COUNT, AVG) dan scanning data dalam jumlah besar karena data disimpan per kolom dengan kompresi tinggi, sehingga hanya kolom yang dibutuhkan saja yang dibaca. Berbeda dengan rowstore index tradisional yang membaca data per baris, membuat columnstore index jauh lebih cepat untuk operasi analitik seperti menghitung total partisipan per semester, rata-rata kehadiran per kegiatan, atau analisis tren partisipasi, yang merupakan kebutuhan umum dalam Data Warehouse.

Step 3: Partitioning Strategy

Tujuan : Memecah tabel fakta berdasarkan Tahun Akademik agar efisien.

```
-- 05_Create_Partitions.sql
```

```
-- 1. Create Partition Function (berdasarkan Tahun Akademik, dimulai Agt)
CREATE PARTITION FUNCTION PF_TahunAkademik_Kemahasiswaan (INT)
AS RANGE RIGHT FOR VALUES
(
    20230801, -- Awal Tahun Akademik 2023/2024

```

```
20240801, -- Awal Tahun Akademik 2024/2025
20250801, -- Awal Tahun Akademik 2025/2026
20260801 -- dan seterusnya...
```

```
);
GO
```

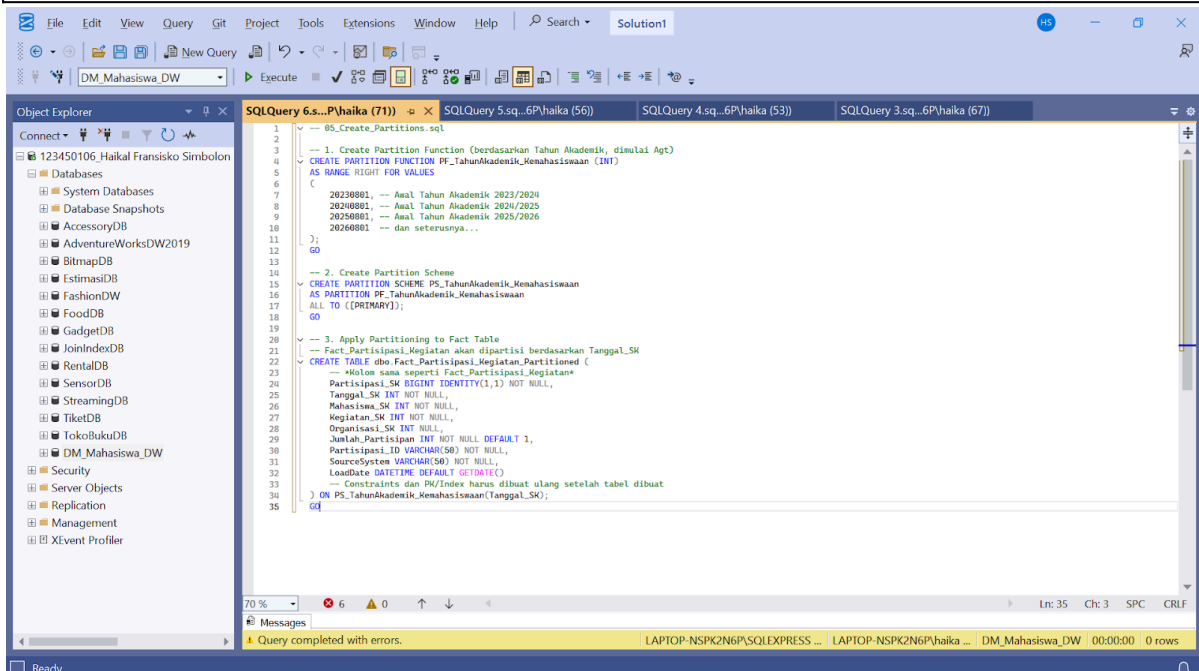
-- 2. Create Partition Scheme

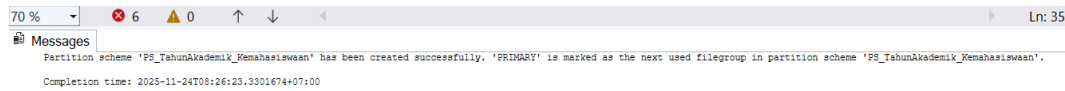
```
CREATE PARTITION SCHEME PS_TahunAkademik_Kemahasiswaan
AS PARTITION PF_TahunAkademik_Kemahasiswaan
ALL TO ([PRIMARY]);
GO
```

-- 3. Apply Partitioning to Fact Table

-- Fact Partisipasi Kegiatan akan dipartisi berdasarkan Tanggal_SK

```
CREATE TABLE dbo.Fact_Partisipasi_Kegiatan_Partitioned (
    -- *Kolom sama seperti Fact_Partisipasi_Kegiatan*
    Partisipasi_SK BIGINT IDENTITY(1,1) NOT NULL,
    Tanggal_SK INT NOT NULL,
    Mahasiswa_SK INT NOT NULL,
    Kegiatan_SK INT NOT NULL,
    Organisasi_SK INT NULL,
    Jumlah_Partisipan INT NOT NULL DEFAULT 1,
    Partisipasi_ID VARCHAR(50) NOT NULL,
    SourceSystem VARCHAR(50) NOT NULL,
    LoadDate DATETIME DEFAULT GETDATE()
    -- Constraints dan PK/Index harus dibuat ulang setelah tabel dibuat
) ON PS_TahunAkademik_Kemahasiswaan(Tanggal_SK);
GO
```





Mengimplementasikan table partitioning untuk meningkatkan performa dan manajemen data dalam Data Warehouse dengan membagi data berdasarkan tahun akademik. Pertama, dibuat partition function bernama PF_TahunAkademik_Kemahasiswaan yang mendefinisikan nilai-nilai batas (boundary values) berdasarkan format integer tanggal (20230801, 20240801, dst) yang merepresentasikan awal tahun akademik setiap bulan Agustus, dengan tipe RANGE RIGHT yang berarti nilai boundary termasuk dalam partisi sebelah kanan. Kemudian dibuat partition scheme bernama PS_TahunAkademik_Kemahasiswaan yang memetakan semua partisi ke filegroup PRIMARY menggunakan kata kunci ALL TO.

Selanjutnya dibuat tabel baru Fact_Partisipasi_Kegiatan_Partitioned dengan struktur kolom yang sama dengan tabel fakta sebelumnya, namun tabel ini diterapkan partitioning dengan klausa ON PS_TahunAkademik_Kemahasiswaan(Tanggal_SK) di akhir statement. Ini berarti data dalam tabel akan secara otomatis dipisahkan ke dalam partisi-partisi berbeda berdasarkan nilai Tanggal_SK sesuai dengan boundary values tahun akademik yang telah didefinisikan. Teknik partitioning ini sangat bermanfaat untuk query yang mengakses data dalam rentang tahun akademik tertentu karena SQL Server hanya perlu membaca partisi yang relevan (partition elimination), mempermudah maintenance seperti archiving data lama, dan meningkatkan performa loading data baru karena setiap partisi dapat dikelola secara independen.

Step 4: ETL Design

Tujuan : Mendesain alur Extract-Transform-Load dari sistem operasional ke DW.

1. ETL Architecture Design

Arsitektur yang digunakan adalah **Staging Area Model**:

- 1) **Source to Staging**: Extract data mentah dari SIAKAD, Sistem Keuangan, atau file CSV ke tabel Staging (Schema stg).
- 2) **Staging to Integration**: Transformasi dan pembersihan data (standarisasi format, penanganan nulls, validasi integritas).
- 3) **Integration to Data Warehouse**: Load data bersih ke Dim dan Fact tables, termasuk penanganan SCD Type 2.

2. Create Staging Tables

```
-- 06_Create_Staging.sql

-- Staging Schema
CREATE SCHEMA stg;
GO

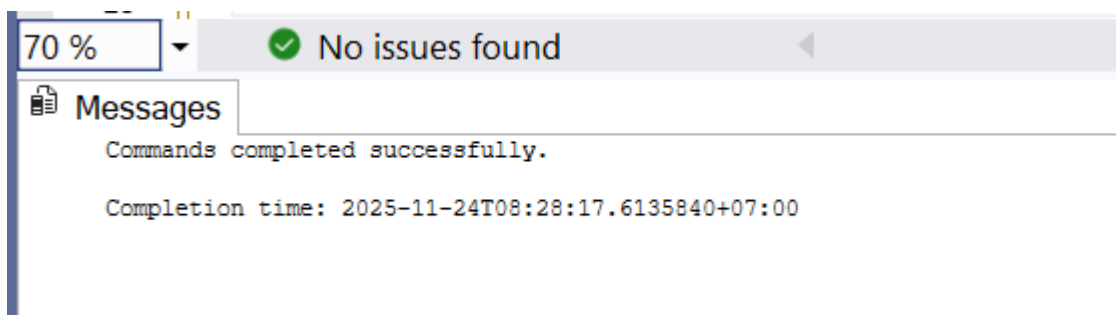
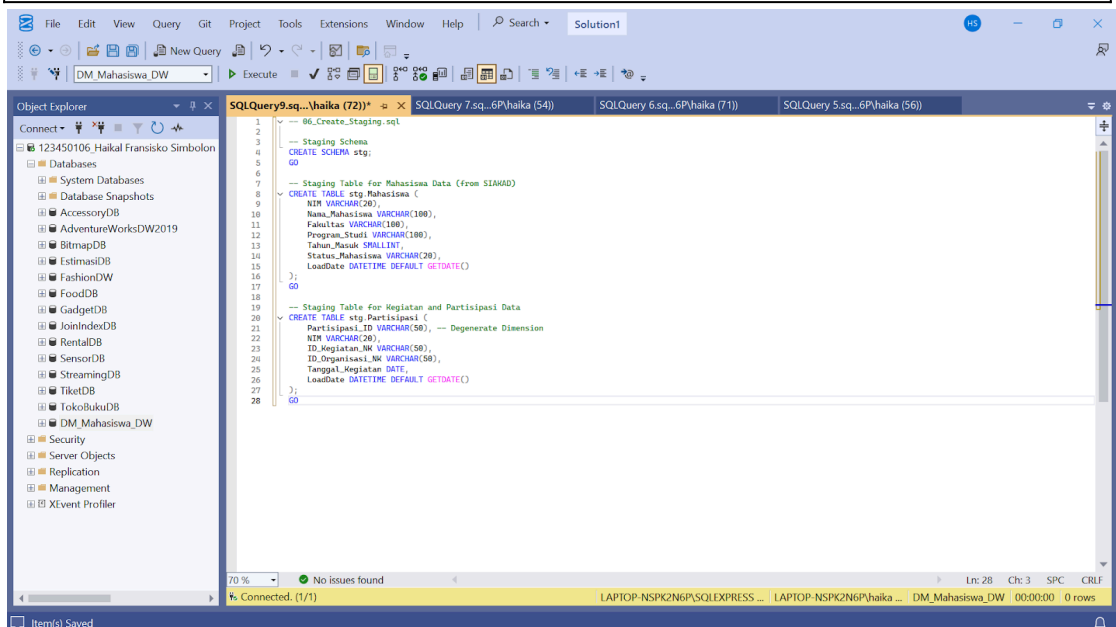
-- Staging Table for Mahasiswa Data (from SIAKAD)
CREATE TABLE stg.Mahasiswa (
```

```

NIM VARCHAR(20),
Nama_Mahasiswa VARCHAR(100),
Fakultas VARCHAR(100),
Program_Studi VARCHAR(100),
Tahun_Masuk SMALLINT,
Status_Mahasiswa VARCHAR(20),
LoadDate DATETIME DEFAULT GETDATE()
);
GO

-- Staging Table for Kegiatan and Partisipasi Data
CREATE TABLE stg.Partisipasi (
    Partisipasi_ID VARCHAR(50), -- Degenerate Dimension
    NIM VARCHAR(20),
    ID_Kegiatan_NK VARCHAR(50),
    ID_Organisasi_NK VARCHAR(50),
    Tanggal_Kegiatan DATE,
    LoadDate DATETIME DEFAULT GETDATE()
);
GO

```



Membuat staging area sebagai area penyimpanan sementara untuk data yang akan dimuat ke Data Warehouse dari sistem sumber OLTP. Pertama dibuat schema khusus

bernama stg (staging) untuk memisahkan dan mengorganisir tabel-tabel staging dari tabel dimensi dan fakta utama. Kemudian dibuat dua tabel staging: stg.Mahasiswa yang berfungsi menampung data mahasiswa sementara dari sistem SIAKAD sebelum diproses dan dimuat ke tabel dimensi Dim_Mahasiswa, dengan kolom-kolom yang mencakup NIM, nama, fakultas, program studi, tahun masuk, dan status mahasiswa, serta kolom LoadDate untuk tracking waktu pemuatan data.

Tabel staging kedua adalah stg.Partisipasi yang menampung data partisipasi mahasiswa dalam kegiatan sebelum ditransformasi ke tabel fakta Fact_Partisipasi_Kegiatan. Tabel ini menyimpan informasi seperti Partisipasi_ID sebagai identifier transaksi, NIM mahasiswa, ID kegiatan dan organisasi (menggunakan natural key dengan suffix _NK), tanggal kegiatan, dan LoadDate. Kedua tabel staging ini sengaja tidak memiliki constraint, index, atau foreign key agar proses loading data dari sistem sumber menjadi lebih cepat dan fleksibel, karena validasi dan transformasi data akan dilakukan pada tahap ETL berikutnya sebelum data akhirnya dimuat ke tabel dimensi dan fakta yang sudah terstruktur.

3. ETL Mapping Document

Source	Source Coloum	Target	Target Coloum	Tranformation
SIAKAD.dbo.mahasiswa	nim	Dim_Mahasiswa	NIM	Direct mapping
SIAKAD.dbo.mahasiswa	nama	Dim_Mahasiswa	Nama_Mahasiswa	UPPER(TRIM(nama))
SIAKAD.dbo.mahasiswa	fakultas_id	Dim_Mahasiswa	Fakultas	Lookup Tabel Fakultas
SIAKAD.dbo.mahasiswa	thn_masuk	Dim_Mahasiswa	Tahun_Masuk	Direct mapping
SIAKAD.dbo.mahasiswa	status	Dim_Mahasiswa	Status_Mahasiswa	CASE WHEN status='A' THEN 'Aktif' ELSE 'Lain-lain' END

Dokumen ETL Mapping mendokumentasikan proses ekstraksi, transformasi, dan loading (ETL) data mahasiswa dari sistem operasional SIAKAD ke dalam data warehouse pada tabel dimensi Dim_Mahasiswa. Proses ini melibatkan transformasi lima atribut data dengan metode yang berbeda-beda sesuai kebutuhan bisnis. Terdapat dua atribut yang menggunakan metode direct mapping atau pemetaan langsung, yaitu NIM dan Tahun_Masuk. Kedua atribut ini tidak mengalami perubahan format atau nilai dari sistem sumber ke sistem target. Atribut Nama_Mahasiswa mengalami

proses transformasi data cleansing menggunakan fungsi UPPER() dan TRIM() untuk menstandarisasi format penulisan menjadi huruf kapital dan menghilangkan spasi yang tidak perlu.

Untuk atribut Fakultas, diterapkan metode lookup transformation yang mengambil referensi berdasarkan fakultas_id, sehingga menghasilkan label nama fakultas yang lebih deskriptif. Sementara itu, atribut Status_Mahasiswa menggunakan transformasi berbasis logika bisnis dengan CASE WHEN statement, mengkonversi kode status 'A' menjadi deskripsi 'Aktif' dan kode lainnya menjadi 'Lain-lain' untuk meningkatkan keterbacaan data dalam proses analisis.

Step 5: ETL Implementation

Tujuan : Mengembangkan proses untuk memuat data dari staging ke DW.

1. Import data ke Staging

```
INSERT INTO dim_mahasiswa (NIM, Nama_Mahasiswa, Fakultas, Program_Studi, Tahun_Masuk, Status_Mahasiswa) VALUES
('202000001', 'Mahasiswa 1', 'Fakultas Sains', 'Matematika', 2020, 'Aktif'),
('202000002', 'Mahasiswa 2', 'Fakultas Teknologi Industri', 'Teknik Industri', 2020, 'Aktif'),
('202000003', 'Mahasiswa 3', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Teknik Sipil', 2020, 'Cuti'),
('202100004', 'Mahasiswa 4', 'Fakultas Sains', 'Biologi', 2021, 'Aktif'),
('202100005', 'Mahasiswa 5', 'Fakultas Teknologi Industri', 'Teknik Telekomunikasi', 2021, 'Aktif'),
('202200006', 'Mahasiswa 6', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Perencanaan Wilayah dan Kota', 2022, 'Aktif'),
('202200007', 'Mahasiswa 7', 'Fakultas Sains', 'Kimia', 2022, 'Aktif'),
('202300008', 'Mahasiswa 8', 'Fakultas Teknologi Industri', 'Teknik Pertambangan', 2023, 'Drop Out'),
('202300009', 'Mahasiswa 9', 'Fakultas Sains', 'Fisika', 2023, 'Aktif'),
('202400010', 'Mahasiswa 10', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Teknik Geomatika', 2024, 'Aktif'),
('202000011', 'Mahasiswa 11', 'Fakultas Sains', 'Farmasi', 2020, 'Aktif'),
('202100012', 'Mahasiswa 12', 'Fakultas Teknologi Industri', 'Teknik Fisika', 2021, 'Aktif'),
('202200013', 'Mahasiswa 13', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Teknik Lingkungan', 2022, 'Aktif'),
('202200014', 'Mahasiswa 14', 'Fakultas Sains', 'Sains Atmosfer dan Keplanetan', 2022, 'Aktif'),
('202300015', 'Mahasiswa 15', 'Fakultas Teknologi Industri', 'Rekayasa Instrumentasi dan Automasi', 2023, 'Cuti'),
('202400016', 'Mahasiswa 16', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Teknik Kelautan', 2024, 'Aktif'),
('202000017', 'Mahasiswa 17', 'Fakultas Sains', 'Sains Lingkungan Kelautan', 2020, 'Aktif'),
('202100018', 'Mahasiswa 18', 'Fakultas Teknologi Industri', 'Teknik Biosistem', 2021, 'Aktif'),
('202200019', 'Mahasiswa 19', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Desain Komunikasi Visual', 2022, 'Aktif'),
('202300020', 'Mahasiswa 20', 'Fakultas Sains', 'Sains Aktuaria', 2023, 'Aktif'),
('202400021', 'Mahasiswa 21', 'Fakultas Teknologi Industri', 'Rekayasa Kehutanan', 2024, 'Aktif'),
```


('202000022', 'Mahasiswa 22', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Arsitektur Lanskap', 2020, 'Drop Out'),
 ('202100023', 'Mahasiswa 23', 'Fakultas Sains', 'Sains Data', 2021, 'Aktif'),
 ('202200024', 'Mahasiswa 24', 'Fakultas Teknologi Industri', 'Rekayasa Keolahragaan', 2022, 'Aktif'),
 ('202300025', 'Mahasiswa 25', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Teknik Perkeretaapian', 2023, 'Aktif'),
 ('202400026', 'Mahasiswa 26', 'Fakultas Sains', 'Biologi', 2024, 'Aktif'),
 ('202000027', 'Mahasiswa 27', 'Fakultas Teknologi Industri', 'Rekayasa Kosmetik', 2020, 'Aktif'),
 ('202100028', 'Mahasiswa 28', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Rekayasa Tata Kelola Air Terpadu', 2021, 'Aktif'),
 ('202200029', 'Mahasiswa 29', 'Fakultas Sains', 'Matematika', 2022, 'Aktif'),
 ('202300030', 'Mahasiswa 30', 'Fakultas Teknologi Industri', 'Rekayasa Minyak dan Gas', 2023, 'Aktif'),
 ('202400031', 'Mahasiswa 31', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Pariwisata', 2024, 'Cuti'),
 ('202000032', 'Mahasiswa 32', 'Fakultas Sains', 'Kimia', 2020, 'Aktif'),
 ('202100033', 'Mahasiswa 33', 'Fakultas Teknologi Industri', 'Teknik Industri', 2021, 'Aktif'),
 ('202200034', 'Mahasiswa 34', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Teknik Sipil', 2022, 'Aktif'),
 ('202300035', 'Mahasiswa 35', 'Fakultas Sains', 'Fisika', 2023, 'Aktif'),
 ('202400036', 'Mahasiswa 36', 'Fakultas Teknologi Industri', 'Teknik Telekomunikasi', 2024, 'Aktif'),
 ('202000037', 'Mahasiswa 37', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Perencanaan Wilayah dan Kota', 2020, 'Aktif'),
 ('202100038', 'Mahasiswa 38', 'Fakultas Sains', 'Farmasi', 2021, 'Aktif'),
 ('202200039', 'Mahasiswa 39', 'Fakultas Teknologi Industri', 'Teknik Pertambangan', 2022, 'Drop Out'),
 ('202300040', 'Mahasiswa 40', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Teknik Geomatika', 2023, 'Aktif'),
 ('202400041', 'Mahasiswa 41', 'Fakultas Sains', 'Sains Data', 2024, 'Aktif'),
 ('202000042', 'Mahasiswa 42', 'Fakultas Teknologi Industri', 'Teknik Fisika', 2020, 'Aktif'),
 ('202100043', 'Mahasiswa 43', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Teknik Lingkungan', 2021, 'Aktif'),
 ('202200044', 'Mahasiswa 44', 'Fakultas Sains', 'Sains Atmosfer dan Keplanetan', 2022, 'Aktif'),
 ('202300045', 'Mahasiswa 45', 'Fakultas Teknologi Industri', 'Rekayasa Instrumentasi dan Automasi', 2023, 'Aktif'),
 ('202400046', 'Mahasiswa 46', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Teknik Kelautan', 2024, 'Cuti'),
 ('202000047', 'Mahasiswa 47', 'Fakultas Sains', 'Sains Lingkungan Kelautan', 2020, 'Aktif'),
 ('202100048', 'Mahasiswa 48', 'Fakultas Teknologi Industri', 'Teknik Biosistem', 2021, 'Aktif'),
 ('202200049', 'Mahasiswa 49', 'Fakultas Teknologi Infrastruktur dan Kewilayahan', 'Desain Komunikasi Visual', 2022, 'Aktif'),
 ('202400050', 'Mahasiswa 50', 'Fakultas Sains', 'Sains Aktuaria', 2024, 'Aktif');

INSERT INTO dim_organisasi (kode_organisasi, nama_organisasi, jenis_organisasi, status)
 VALUES

('ORG001', 'BEM ITERA', 'Ormawa', 'Aktif'),
 ('ORG002', 'BEM Fakultas Sains', 'Ormawa', 'Aktif'),
 ('ORG003', 'BEM FTIK', 'Ormawa', 'Aktif'),
 ('ORG004', 'BEM FTI', 'Ormawa', 'Aktif'),
 ('ORG005', 'HIMASAINS (Sains Data)', 'Ormawa', 'Aktif'),
 ('ORG006', 'HIMBIO (Biologi)', 'Ormawa', 'Aktif'),
 ('ORG007', 'HIMAKI (Kimia)', 'Ormawa', 'Aktif'),
 ('ORG008', 'HIMATIKA (Matematika)', 'Ormawa', 'Aktif'),
 ('ORG009', 'HIMAFI (Fisika)', 'Ormawa', 'Aktif'),
 ('ORG010', 'HIMAFAR (Farmasi)', 'Ormawa', 'Aktif'),
 ('ORG011', 'HIMASIP (Teknik Sipil)', 'Ormawa', 'Aktif'),
 ('ORG012', 'HIMAPWK (PWK)', 'Ormawa', 'Aktif'),
 ('ORG013', 'UKM Basket', 'UKM', 'Aktif'),
 ('ORG014', 'UKM Futsal', 'UKM', 'Aktif'),
 ('ORG015', 'UKM Badminton', 'UKM', 'Aktif'),
 ('ORG016', 'UKM Musik', 'UKM', 'Aktif'),
 ('ORG017', 'UKM Teater', 'UKM', 'Aktif'),
 ('ORG018', 'UKM Fotografi', 'UKM', 'Aktif'),
 ('ORG019', 'UKM Robotika', 'UKM', 'Aktif'),
 ('ORG020', 'UKM Sinematografi', 'UKM', 'Aktif'),
 ('ORG021', 'UKM Pramuka', 'UKM', 'Aktif'),
 ('ORG022', 'UKM PMI', 'UKM', 'Aktif'),
 ('ORG023', 'UKM KSR', 'UKM', 'Aktif'),
 ('ORG024', 'UKM Menwa', 'UKM', 'Aktif'),
 ('ORG025', 'UKM Mapala', 'UKM', 'Aktif'),
 ('ORG026', 'UKM Taekwondo', 'UKM', 'Aktif'),
 ('ORG027', 'UKM English Club', 'UKM', 'Aktif'),
 ('ORG028', 'UKM Entrepreneur', 'UKM', 'Aktif'),
 ('ORG029', 'UKM Science Club', 'UKM', 'Aktif'),
 ('ORG030', 'UKM Coding Club', 'UKM', 'Aktif');

INSERT INTO dim_kegiatan (id_kegiatan, nama_kegiatan, jenis_kegiatan, skala_kegiatan, status_kegiatan) VALUES

('KGT00001', 'Seminar 1', 'Seminar', 'Internal', 'Terlaksana'),
 ('KGT00002', 'Workshop 2', 'Workshop', 'Nasional', 'Terlaksana'),
 ('KGT00003', 'Lomba 3', 'Lomba', 'Internasional', 'Terlaksana'),
 ('KGT00004', 'Bakti Sosial 4', 'Bakti Sosial', 'Internal', 'Terlaksana'),
 ('KGT00005', 'Pelatihan 5', 'Pelatihan', 'Nasional', 'Batal'),
 ('KGT00006', 'Kompetisi 6', 'Kompetisi', 'Internasional', 'Terlaksana'),
 ('KGT00007', 'Seminar 7', 'Seminar', 'Internal', 'Terlaksana'),
 ('KGT00008', 'Workshop 8', 'Workshop', 'Nasional', 'Terlaksana'),
 ('KGT00009', 'Lomba 9', 'Lomba', 'Internal', 'Terlaksana'),
 ('KGT00010', 'Bakti Sosial 10', 'Bakti Sosial', 'Nasional', 'Terlaksana'),
 ('KGT00011', 'Pelatihan 11', 'Pelatihan', 'Internasional', 'Terlaksana'),
 ('KGT00012', 'Kompetisi 12', 'Kompetisi', 'Internal', 'Terlaksana'),
 ('KGT00013', 'Seminar 13', 'Seminar', 'Nasional', 'Terlaksana'),
 ('KGT00014', 'Workshop 14', 'Workshop', 'Internasional', 'Terlaksana'),

('KGT00015', 'Lomba 15', 'Lomba', 'Internal', 'Terlaksana'),
 ('KGT00016', 'Bakti Sosial 16', 'Bakti Sosial', 'Nasional', 'Terlaksana'),
 ('KGT00017', 'Pelatihan 17', 'Pelatihan', 'Internasional', 'Terlaksana'),
 ('KGT00018', 'Kompetisi 18', 'Kompetisi', 'Internal', 'Batal'),
 ('KGT00019', 'Seminar 19', 'Seminar', 'Nasional', 'Terlaksana'),
 ('KGT00020', 'Workshop 20', 'Workshop', 'Internasional', 'Terlaksana'),
 ('KGT00021', 'Lomba 21', 'Lomba', 'Internal', 'Terlaksana'),
 ('KGT00022', 'Bakti Sosial 22', 'Bakti Sosial', 'Nasional', 'Terlaksana'),
 ('KGT00023', 'Pelatihan 23', 'Pelatihan', 'Internasional', 'Terlaksana'),
 ('KGT00024', 'Kompetisi 24', 'Kompetisi', 'Internal', 'Terlaksana'),
 ('KGT00025', 'Seminar 25', 'Seminar', 'Nasional', 'Terlaksana'),
 ('KGT00026', 'Workshop 26', 'Workshop', 'Internasional', 'Terlaksana'),
 ('KGT00027', 'Lomba 27', 'Lomba', 'Internal', 'Terlaksana'),
 ('KGT00028', 'Bakti Sosial 28', 'Bakti Sosial', 'Nasional', 'Terlaksana'),
 ('KGT00029', 'Pelatihan 29', 'Pelatihan', 'Internasional', 'Terlaksana'),
 ('KGT00030', 'Kompetisi 30', 'Kompetisi', 'Internal', 'Terlaksana'),
 ('KGT00031', 'Seminar 31', 'Seminar', 'Nasional', 'Batal'),
 ('KGT00032', 'Workshop 32', 'Workshop', 'Internasional', 'Terlaksana'),
 ('KGT00033', 'Lomba 33', 'Lomba', 'Internal', 'Terlaksana'),
 ('KGT00034', 'Bakti Sosial 34', 'Bakti Sosial', 'Nasional', 'Terlaksana'),
 ('KGT00035', 'Pelatihan 35', 'Pelatihan', 'Internasional', 'Terlaksana'),
 ('KGT00036', 'Kompetisi 36', 'Kompetisi', 'Internal', 'Terlaksana'),
 ('KGT00037', 'Seminar 37', 'Seminar', 'Nasional', 'Terlaksana'),
 ('KGT00038', 'Workshop 38', 'Workshop', 'Internasional', 'Terlaksana'),
 ('KGT00039', 'Lomba 39', 'Lomba', 'Internal', 'Terlaksana'),
 ('KGT00040', 'Bakti Sosial 40', 'Bakti Sosial', 'Nasional', 'Terlaksana'),
 ('KGT00041', 'Pelatihan 41', 'Pelatihan', 'Internasional', 'Terlaksana'),
 ('KGT00042', 'Kompetisi 42', 'Kompetisi', 'Internal', 'Terlaksana'),
 ('KGT00043', 'Seminar 43', 'Seminar', 'Nasional', 'Terlaksana'),
 ('KGT00044', 'Workshop 44', 'Workshop', 'Internasional', 'Terlaksana'),
 ('KGT00045', 'Lomba 45', 'Lomba', 'Internal', 'Terlaksana'),
 ('KGT00046', 'Bakti Sosial 46', 'Bakti Sosial', 'Nasional', 'Terlaksana'),
 ('KGT00047', 'Pelatihan 47', 'Pelatihan', 'Internasional', 'Terlaksana'),
 ('KGT00048', 'Kompetisi 48', 'Kompetisi', 'Internal', 'Terlaksana'),
 ('KGT00049', 'Seminar 49', 'Seminar', 'Nasional', 'Terlaksana'),
 ('KGT00050', 'Workshop 50', 'Workshop', 'Internasional', 'Terlaksana');

INSERT INTO dim_prestasi (ID_Prestasi, Nama_Penghargaan, Level_Prestasi, Kategori_Prestasi)
 VALUES

('PRE00001', 'Penghargaan Nasional 1', 'Nasional', 'Akademik'),
 ('PRE00002', 'Penghargaan Internasional 2', 'Internasional', 'Non-Akademik'),
 ('PRE00003', 'Penghargaan Lokal 3', 'Lokal', 'Akademik'),
 ('PRE00004', 'Penghargaan Regional 4', 'Regional', 'Non-Akademik'),
 ('PRE00005', 'Penghargaan Nasional 5', 'Nasional', 'Akademik'),
 ('PRE00006', 'Penghargaan Internasional 6', 'Internasional', 'Non-Akademik'),
 ('PRE00007', 'Penghargaan Lokal 7', 'Lokal', 'Akademik'),

('PRE00008', 'Penghargaan Regional 8', 'Regional', 'Non-Akademik'),
 ('PRE00009', 'Penghargaan Nasional 9', 'Nasional', 'Akademik'),
 ('PRE00010', 'Penghargaan Internasional 10', 'Internasional', 'Non-Akademik'),
 ('PRE00011', 'Penghargaan Lokal 11', 'Lokal', 'Akademik'),
 ('PRE00012', 'Penghargaan Regional 12', 'Regional', 'Non-Akademik'),
 ('PRE00013', 'Penghargaan Nasional 13', 'Nasional', 'Akademik'),
 ('PRE00014', 'Penghargaan Internasional 14', 'Internasional', 'Non-Akademik'),
 ('PRE00015', 'Penghargaan Lokal 15', 'Lokal', 'Akademik'),
 ('PRE00016', 'Penghargaan Regional 16', 'Regional', 'Non-Akademik'),
 ('PRE00017', 'Penghargaan Nasional 17', 'Nasional', 'Akademik'),
 ('PRE00018', 'Penghargaan Internasional 18', 'Internasional', 'Non-Akademik'),
 ('PRE00019', 'Penghargaan Lokal 19', 'Lokal', 'Akademik'),
 ('PRE00020', 'Penghargaan Regional 20', 'Regional', 'Non-Akademik'),
 ('PRE00021', 'Penghargaan Nasional 21', 'Nasional', 'Akademik'),
 ('PRE00022', 'Penghargaan Internasional 22', 'Internasional', 'Non-Akademik'),
 ('PRE00023', 'Penghargaan Lokal 23', 'Lokal', 'Akademik'),
 ('PRE00024', 'Penghargaan Regional 24', 'Regional', 'Non-Akademik'),
 ('PRE00025', 'Penghargaan Nasional 25', 'Nasional', 'Akademik'),
 ('PRE00026', 'Penghargaan Internasional 26', 'Internasional', 'Non-Akademik'),
 ('PRE00027', 'Penghargaan Lokal 27', 'Lokal', 'Akademik'),
 ('PRE00028', 'Penghargaan Regional 28', 'Regional', 'Non-Akademik'),
 ('PRE00029', 'Penghargaan Nasional 29', 'Nasional', 'Akademik'),
 ('PRE00030', 'Penghargaan Internasional 30', 'Internasional', 'Non-Akademik'),
 ('PRE00031', 'Penghargaan Lokal 31', 'Lokal', 'Akademik'),
 ('PRE00032', 'Penghargaan Regional 32', 'Regional', 'Non-Akademik'),
 ('PRE00033', 'Penghargaan Nasional 33', 'Nasional', 'Akademik'),
 ('PRE00034', 'Penghargaan Internasional 34', 'Internasional', 'Non-Akademik'),
 ('PRE00035', 'Penghargaan Lokal 35', 'Lokal', 'Akademik'),
 ('PRE00036', 'Penghargaan Regional 36', 'Regional', 'Non-Akademik'),
 ('PRE00037', 'Penghargaan Nasional 37', 'Nasional', 'Akademik'),
 ('PRE00038', 'Penghargaan Internasional 38', 'Internasional', 'Non-Akademik'),
 ('PRE00039', 'Penghargaan Lokal 39', 'Lokal', 'Akademik'),
 ('PRE00040', 'Penghargaan Regional 40', 'Regional', 'Non-Akademik'),
 ('PRE00041', 'Penghargaan Nasional 41', 'Nasional', 'Akademik'),
 ('PRE00042', 'Penghargaan Internasional 42', 'Internasional', 'Non-Akademik'),
 ('PRE00043', 'Penghargaan Lokal 43', 'Lokal', 'Akademik'),
 ('PRE00044', 'Penghargaan Regional 44', 'Regional', 'Non-Akademik'),
 ('PRE00045', 'Penghargaan Nasional 45', 'Nasional', 'Akademik'),
 ('PRE00046', 'Penghargaan Internasional 46', 'Internasional', 'Non-Akademik'),
 ('PRE00047', 'Penghargaan Lokal 47', 'Lokal', 'Akademik'),
 ('PRE00048', 'Penghargaan Regional 48', 'Regional', 'Non-Akademik'),
 ('PRE00049', 'Penghargaan Nasional 49', 'Nasional', 'Akademik'),
 ('PRE00050', 'Penghargaan Internasional 50', 'Internasional', 'Non-Akademik');

INSERT INTO dim_beasiswa (ID_Beasiswa, Nama_Beasiswa, Sumber_Dana, Periode_Berlaku)
 VALUES

('BEA001', 'Beasiswa PPA', 'Internal', '2023-2024'),
 ('BEA002', 'Beasiswa BBM', 'Internal', '2023-2024'),
 ('BEA003', 'Beasiswa Prestasi', 'Internal', '2023-2024'),
 ('BEA004', 'Beasiswa KIP Kuliah', 'Eksternal', '2023-2024'),
 ('BEA005', 'Beasiswa Bank Indonesia', 'Eksternal', '2023-2024'),
 ('BEA006', 'Beasiswa Djarum', 'Eksternal', '2023-2024'),
 ('BEA007', 'Beasiswa Tanoto Foundation', 'Eksternal', '2023-2024'),
 ('BEA008', 'Beasiswa LPDP', 'Eksternal', '2023-2024'),
 ('BEA009', 'Beasiswa Unggulan', 'Eksternal', '2023-2024'),
 ('BEA010', 'Beasiswa Chevron', 'Eksternal', '2023-2024'),
 ('BEA011', 'Beasiswa Pertamina', 'Eksternal', '2024-2025'),
 ('BEA012', 'Beasiswa BCA', 'Eksternal', '2024-2025'),
 ('BEA013', 'Beasiswa Mandiri', 'Eksternal', '2024-2025'),
 ('BEA014', 'Beasiswa BNI', 'Eksternal', '2024-2025'),
 ('BEA015', 'Beasiswa Telkom', 'Eksternal', '2024-2025'),
 ('BEA016', 'Beasiswa Indosat', 'Eksternal', '2024-2025'),
 ('BEA017', 'Beasiswa XL Axiata', 'Eksternal', '2024-2025'),
 ('BEA018', 'Beasiswa Astra', 'Eksternal', '2024-2025'),
 ('BEA019', 'Beasiswa Sampoerna', 'Eksternal', '2024-2025'),
 ('BEA020', 'Beasiswa Orbit', 'Eksternal', '2024-2025');

INSERT INTO dbo.Dim_Tanggal (Tanggal_SK, FullDate, DayNumberOfWeek, DayName,
 MonthName, MonthNumber, Quarter, QuarterName, Tahun, IsWeekend, Semester) VALUES
 (20230311, '2023-03-11', 7, 'Saturday', 'March', 3, 1, 'Q1', 2023, 1, 'Ganjil'),
 (20230315, '2023-03-15', 3, 'Wednesday', 'March', 3, 1, 'Q1', 2023, 0, 'Ganjil'),
 (20230406, '2023-04-06', 4, 'Thursday', 'April', 4, 2, 'Q2', 2023, 0, 'Genap'),
 (20230411, '2023-04-11', 2, 'Tuesday', 'April', 4, 2, 'Q2', 2023, 0, 'Genap'),
 (20230503, '2023-05-03', 3, 'Wednesday', 'May', 5, 2, 'Q2', 2023, 0, 'Genap'),
 (20230514, '2023-05-14', 1, 'Sunday', 'May', 5, 2, 'Q2', 2023, 1, 'Genap'),
 (20230523, '2023-05-23', 2, 'Tuesday', 'May', 5, 2, 'Q2', 2023, 0, 'Genap'),
 (20230526, '2023-05-26', 5, 'Friday', 'May', 5, 2, 'Q2', 2023, 0, 'Genap'),
 (20230601, '2023-06-01', 4, 'Thursday', 'June', 6, 2, 'Q2', 2023, 0, 'Genap'),
 (20230620, '2023-06-20', 2, 'Tuesday', 'June', 6, 2, 'Q2', 2023, 0, 'Genap'),
 (20230627, '2023-06-27', 3, 'Wednesday', 'June', 6, 2, 'Q2', 2023, 0, 'Genap'),
 (20230630, '2023-06-30', 5, 'Friday', 'June', 6, 2, 'Q2', 2023, 0, 'Genap'),
 (20230705, '2023-07-05', 3, 'Wednesday', 'July', 7, 3, 'Q3', 2023, 0, 'Ganjil'),
 (20230719, '2023-07-19', 3, 'Wednesday', 'July', 7, 3, 'Q3', 2023, 0, 'Ganjil'),
 (20230728, '2023-07-28', 5, 'Friday', 'July', 7, 3, 'Q3', 2023, 0, 'Ganjil'),
 (20230730, '2023-07-30', 7, 'Sunday', 'July', 7, 3, 'Q3', 2023, 1, 'Ganjil'),
 (20230809, '2023-08-09', 3, 'Wednesday', 'August', 8, 3, 'Q3', 2023, 0, 'Ganjil'),
 (20230815, '2023-08-15', 2, 'Tuesday', 'August', 8, 3, 'Q3', 2023, 0, 'Ganjil'),
 (20230820, '2023-08-20', 7, 'Sunday', 'August', 8, 3, 'Q3', 2023, 1, 'Ganjil'),
 (20230829, '2023-08-29', 2, 'Tuesday', 'August', 8, 3, 'Q3', 2023, 0, 'Ganjil'),
 (20230903, '2023-09-03', 7, 'Sunday', 'September', 9, 3, 'Q3', 2023, 1, 'Ganjil'),
 (20230907, '2023-09-07', 4, 'Thursday', 'September', 9, 3, 'Q3', 2023, 0, 'Ganjil'),
 (20230908, '2023-09-08', 5, 'Friday', 'September', 9, 3, 'Q3', 2023, 0, 'Ganjil'),
 (20230913, '2023-09-13', 3, 'Wednesday', 'September', 9, 3, 'Q3', 2023, 0, 'Ganjil'),

(20231002, '2023-10-02', 1, 'Monday', 'October', 10, 4, 'Q4', 2023, 0, 'Ganjil'),
(20231006, '2023-10-06', 5, 'Friday', 'October', 10, 4, 'Q4', 2023, 0, 'Ganjil'),
(20231014, '2023-10-14', 7, 'Saturday', 'October', 10, 4, 'Q4', 2023, 1, 'Ganjil'),
(20231022, '2023-10-22', 1, 'Sunday', 'October', 10, 4, 'Q4', 2023, 1, 'Ganjil'),
(20231025, '2023-10-25', 3, 'Wednesday', 'October', 10, 4, 'Q4', 2023, 0, 'Ganjil'),
(20231104, '2023-11-04', 7, 'Saturday', 'November', 11, 4, 'Q4', 2023, 1, 'Ganjil'),
(20231112, '2023-11-12', 1, 'Sunday', 'November', 11, 4, 'Q4', 2023, 1, 'Ganjil'),
(20231123, '2023-11-23', 4, 'Thursday', 'November', 11, 4, 'Q4', 2023, 0, 'Ganjil'),
(20231127, '2023-11-27', 1, 'Monday', 'November', 11, 4, 'Q4', 2023, 0, 'Ganjil'),
(20231130, '2023-11-30', 4, 'Thursday', 'November', 11, 4, 'Q4', 2023, 0, 'Ganjil'),
(20231205, '2023-12-05', 2, 'Tuesday', 'December', 12, 4, 'Q4', 2023, 0, 'Ganjil'),
(20231218, '2023-12-18', 1, 'Monday', 'December', 12, 4, 'Q4', 2023, 0, 'Ganjil'),
(20231226, '2023-12-26', 2, 'Tuesday', 'December', 12, 4, 'Q4', 2023, 0, 'Ganjil'),
(20231227, '2023-12-27', 3, 'Wednesday', 'December', 12, 4, 'Q4', 2023, 0, 'Ganjil'),
(20231230, '2023-12-30', 6, 'Saturday', 'December', 12, 4, 'Q4', 2023, 1, 'Ganjil'),
(20240105, '2024-01-05', 5, 'Friday', 'January', 1, 1, 'Q1', 2024, 0, 'Genap'),
(20240108, '2024-01-08', 1, 'Monday', 'January', 1, 1, 'Q1', 2024, 0, 'Genap'),
(20240110, '2024-01-10', 3, 'Wednesday', 'January', 1, 1, 'Q1', 2024, 0, 'Genap'),
(20240116, '2024-01-16', 2, 'Tuesday', 'January', 1, 1, 'Q1', 2024, 0, 'Genap'),
(20240117, '2024-01-17', 3, 'Wednesday', 'January', 1, 1, 'Q1', 2024, 0, 'Genap'),
(20240128, '2024-01-28', 1, 'Sunday', 'January', 1, 1, 'Q1', 2024, 1, 'Genap'),
(20240204, '2024-02-04', 1, 'Sunday', 'February', 2, 1, 'Q1', 2024, 1, 'Genap'),
(20240207, '2024-02-07', 3, 'Wednesday', 'February', 2, 1, 'Q1', 2024, 0, 'Genap'),
(20240212, '2024-02-12', 1, 'Monday', 'February', 2, 1, 'Q1', 2024, 0, 'Genap'),
(20240214, '2024-02-14', 3, 'Wednesday', 'February', 2, 1, 'Q1', 2024, 0, 'Genap'),
(20240226, '2024-02-26', 1, 'Monday', 'February', 2, 1, 'Q1', 2024, 0, 'Genap'),
(20240228, '2024-02-28', 3, 'Wednesday', 'February', 2, 1, 'Q1', 2024, 0, 'Genap'),
(20240306, '2024-03-06', 4, 'Wednesday', 'March', 3, 1, 'Q1', 2024, 0, 'Genap'),
(20240309, '2024-03-09', 7, 'Saturday', 'March', 3, 1, 'Q1', 2024, 1, 'Genap'),
(20240310, '2024-03-10', 1, 'Sunday', 'March', 3, 1, 'Q1', 2024, 1, 'Genap'),
(20240312, '2024-03-12', 3, 'Tuesday', 'March', 3, 1, 'Q1', 2024, 0, 'Genap'),
(20240319, '2024-03-19', 3, 'Tuesday', 'March', 3, 1, 'Q1', 2024, 0, 'Genap'),
(20240325, '2024-03-25', 1, 'Monday', 'March', 3, 1, 'Q1', 2024, 0, 'Genap'),
(20240402, '2024-04-02', 3, 'Tuesday', 'April', 4, 2, 'Q2', 2024, 0, 'Genap'),
(20240416, '2024-04-16', 3, 'Tuesday', 'April', 4, 2, 'Q2', 2024, 0, 'Genap'),
(20240418, '2024-04-18', 4, 'Thursday', 'April', 4, 2, 'Q2', 2024, 0, 'Genap'),
(20240420, '2024-04-20', 6, 'Saturday', 'April', 4, 2, 'Q2', 2024, 1, 'Genap'),
(20240422, '2024-04-22', 1, 'Monday', 'April', 4, 2, 'Q2', 2024, 0, 'Genap'),
(20240427, '2024-04-27', 6, 'Saturday', 'April', 4, 2, 'Q2', 2024, 1, 'Genap'),
(20240511, '2024-05-11', 6, 'Saturday', 'May', 5, 2, 'Q2', 2024, 1, 'Genap'),
(20240515, '2024-05-15', 3, 'Wednesday', 'May', 5, 2, 'Q2', 2024, 0, 'Genap'),
(20240524, '2024-05-24', 5, 'Friday', 'May', 5, 2, 'Q2', 2024, 0, 'Genap'),
(20240530, '2024-05-30', 4, 'Thursday', 'May', 5, 2, 'Q2', 2024, 0, 'Genap'),
(20240607, '2024-06-07', 5, 'Friday', 'June', 6, 2, 'Q2', 2024, 0, 'Genap'),
(20240611, '2024-06-11', 2, 'Tuesday', 'June', 6, 2, 'Q2', 2024, 0, 'Genap'),
(20240618, '2024-06-18', 2, 'Tuesday', 'June', 6, 2, 'Q2', 2024, 0, 'Genap'),
(20240621, '2024-06-21', 5, 'Friday', 'June', 6, 2, 'Q2', 2024, 0, 'Genap'),

(20240622, '2024-06-22', 6, 'Saturday', 'June', 6, 2, 'Q2', 2024, 1, 'Genap'),
 (20240704, '2024-07-04', 4, 'Thursday', 'July', 7, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240715, '2024-07-15', 1, 'Monday', 'July', 7, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240719, '2024-07-19', 5, 'Friday', 'July', 7, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240726, '2024-07-26', 5, 'Friday', 'July', 7, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240808, '2024-08-08', 4, 'Thursday', 'August', 8, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240813, '2024-08-13', 2, 'Tuesday', 'August', 8, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240814, '2024-08-14', 3, 'Wednesday', 'August', 8, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240817, '2024-08-17', 6, 'Saturday', 'August', 8, 3, 'Q3', 2024, 1, 'Ganjil'),
 (20240829, '2024-08-29', 4, 'Thursday', 'August', 8, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240831, '2024-08-31', 6, 'Saturday', 'August', 8, 3, 'Q3', 2024, 1, 'Ganjil'),
 (20240903, '2024-09-03', 2, 'Tuesday', 'September', 9, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240908, '2024-09-08', 1, 'Sunday', 'September', 9, 3, 'Q3', 2024, 1, 'Ganjil'),
 (20240911, '2024-09-11', 4, 'Wednesday', 'September', 9, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240923, '2024-09-23', 1, 'Monday', 'September', 9, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20240924, '2024-09-24', 3, 'Tuesday', 'September', 9, 3, 'Q3', 2024, 0, 'Ganjil'),
 (20241011, '2024-10-11', 5, 'Friday', 'October', 10, 4, 'Q4', 2024, 0, 'Ganjil'),
 (20241021, '2024-10-21', 1, 'Monday', 'October', 10, 4, 'Q4', 2024, 0, 'Ganjil'),
 (20241023, '2024-10-23', 3, 'Wednesday', 'October', 10, 4, 'Q4', 2024, 0, 'Ganjil'),
 (20241103, '2024-11-03', 1, 'Sunday', 'November', 11, 4, 'Q4', 2024, 1, 'Ganjil'),
 (20241110, '2024-11-10', 1, 'Sunday', 'November', 11, 4, 'Q4', 2024, 1, 'Ganjil'),
 (20241115, '2024-11-15', 5, 'Friday', 'November', 11, 4, 'Q4', 2024, 0, 'Ganjil'),
 (20241119, '2024-11-19', 3, 'Tuesday', 'November', 11, 4, 'Q4', 2024, 0, 'Ganjil'),
 (20241215, '2024-12-15', 1, 'Sunday', 'December', 12, 4, 'Q4', 2024, 1, 'Ganjil');

INSERT INTO dbo.Fact_Partisipasi_Kegiatan (Tanggal_SK, Mahasiswa_SK, Kegiatan_SK, Organisasi_SK, Jumlah_Partisipan, Partisipasi_ID, SourceSystem)

VALUES

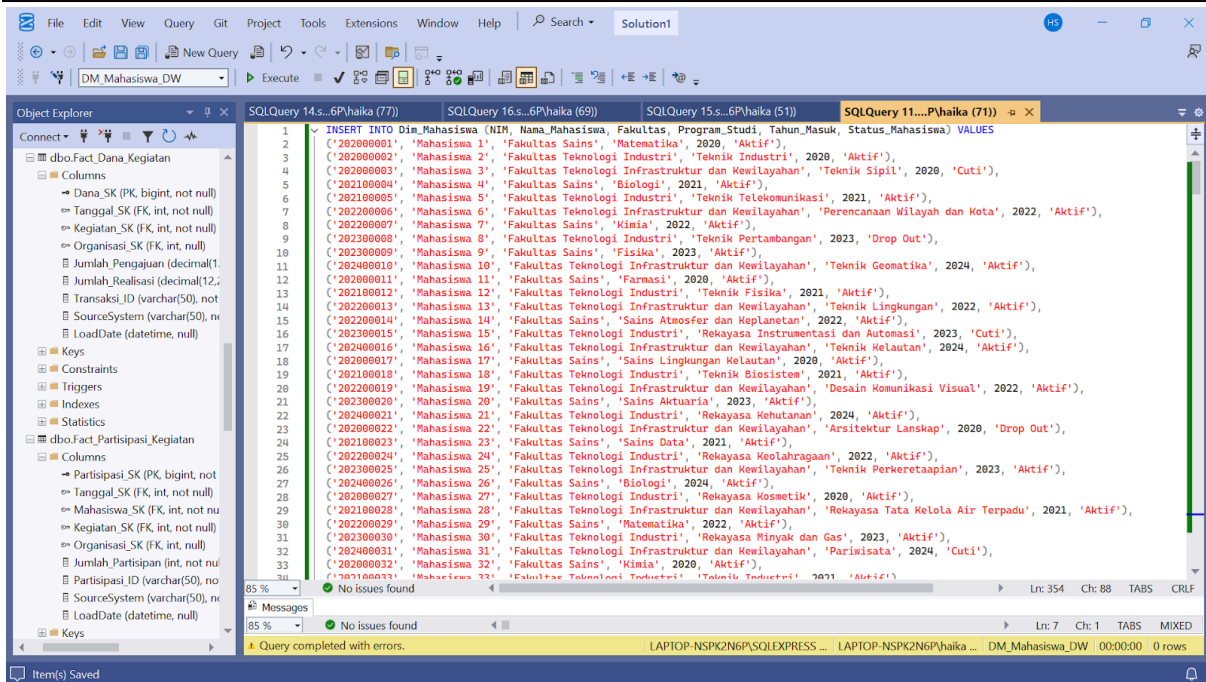
(20230315, 1, 1, 1, 201, 'PART0000000001', 'OLTP'),
 (20230620, 4, 2, 2, 204, 'PART0000000002', 'OLTP'),
 (20240110, 6, 3, 3, 206, 'PART0000000003', 'OLTP'),
 (20240422, 8, 4, 4, 208, 'PART0000000004', 'OLTP'),
 (20240715, 10, 5, 5, 210, 'PART0000000005', 'OLTP'),
 (20230908, 11, 6, 6, 211, 'PART0000000006', 'OLTP'),
 (20240214, 12, 7, 7, 212, 'PART0000000007', 'OLTP'),
 (20240530, 13, 8, 8, 2013, 'PART0000000008', 'OLTP'),
 (20231112, 14, 9, 20, 2114, 'PART0000000009', 'OLTP'),
 (20240325, 15, 10, 10, 225, 'PART0000000010', 'OLTP'),
 (20240817, 16, 11, 11, 206, 'PART0000000011', 'OLTP'),
 (20231205, 17, 12, 9, 227, 'PART0000000012', 'OLTP'),
 (20240128, 18, 13, 12, 208, 'PART0000000013', 'OLTP'),
 (20240611, 19, 14, 13, 219, 'PART0000000014', 'OLTP'),
 (20230719, 20, 15, 14, 200, 'PART0000000015', 'OLTP'),
 (20240903, 21, 16, 15, 231, 'PART0000000016', 'OLTP'),
 (20231022, 22, 17, 20, 202, 'PART0000000017', 'OLTP'),
 (20240207, 23, 18, 16, 221, 'PART0000000018', 'OLTP'),
 (20240416, 24, 19, 17, 244, 'PART0000000019', 'OLTP'),

(20230829, 25, 20, 18, 235, 'PART0000000020', 'OLTP'),
(20241110, 26, 21, 19, 245, 'PART0000000021', 'OLTP'),
(20230514, 27, 22, 21, 227, 'PART0000000022', 'OLTP'),
(20240726, 28, 23, 22, 228, 'PART0000000023', 'OLTP'),
(20231218, 29, 24, 23, 229, 'PART0000000024', 'OLTP'),
(20240309, 30, 25, 24, 230, 'PART0000000025', 'OLTP'),
(20240621, 31, 26, 20, 201, 'PART0000000026', 'OLTP'),
(20230913, 32, 27, 25, 232, 'PART0000000027', 'OLTP'),
(20240105, 33, 28, 26, 203, 'PART0000000028', 'OLTP'),
(20240418, 34, 29, 1, 234, 'PART0000000029', 'OLTP'),
(20231127, 35, 30, 27, 335, 'PART0000000030', 'OLTP');
GO

INSERT INTO dbo.Fact_Dana_Kegiatan (Tanggal_SK, Kegiatan_SK, Organisasi_SK, Jumlah_Pengajuan, Jumlah_Realisasi, Transaksi_ID, SourceSystem) VALUES

(20230315, 1, 1, 15000000.00, 12500000.00, 'TRX0000000001', 'OLTP'),
(20230620, 2, 2, 25000000.00, 22000000.00, 'TRX0000000002', 'OLTP'),
(20240110, 3, 3, 35000000.00, 30500000.00, 'TRX0000000003', 'OLTP'),
(20240422, 4, 4, 10000000.00, 8500000.00, 'TRX0000000004', 'OLTP'),
(20240715, 5, 5, 18000000.00, 15800000.00, 'TRX0000000005', 'OLTP'),
(20230908, 6, 6, 42000000.00, 37500000.00, 'TRX0000000006', 'OLTP'),
(20240214, 7, 7, 20000000.00, 17200000.00, 'TRX0000000007', 'OLTP'),
(20240530, 8, 8, 12000000.00, 10500000.00, 'TRX0000000008', 'OLTP'),
(20231112, 9, 9, 28000000.00, 24000000.00, 'TRX0000000009', 'OLTP'),
(20240325, 10, 10, 8000000.00, 7000000.00, 'TRX0000000010', 'OLTP'),
(20240817, 11, 11, 22000000.00, 19500000.00, 'TRX0000000011', 'OLTP'),
(20231205, 12, 9, 38000000.00, 33000000.00, 'TRX0000000012', 'OLTP'),
(20240128, 13, 12, 16000000.00, 14000000.00, 'TRX0000000013', 'OLTP'),
(20240611, 14, 13, 14000000.00, 12300000.00, 'TRX0000000014', 'OLTP'),
(20230719, 15, 14, 30000000.00, 26500000.00, 'TRX0000000015', 'OLTP'),
(20240903, 16, 15, 11000000.00, 9800000.00, 'TRX0000000016', 'OLTP'),
(20231022, 17, 16, 19000000.00, 16700000.00, 'TRX0000000017', 'OLTP'),
(20240207, 18, 16, 45000000.00, 40000000.00, 'TRX0000000018', 'OLTP'),
(20240416, 19, 17, 17000000.00, 15000000.00, 'TRX0000000019', 'OLTP'),
(20230829, 20, 18, 13000000.00, 11500000.00, 'TRX0000000020', 'OLTP'),
(20241110, 21, 19, 32000000.00, 28000000.00, 'TRX0000000021', 'OLTP'),
(20230514, 22, 20, 9000000.00, 7900000.00, 'TRX0000000022', 'OLTP'),
(20240726, 23, 21, 21000000.00, 18500000.00, 'TRX0000000023', 'OLTP'),
(20231218, 24, 22, 40000000.00, 35500000.00, 'TRX0000000024', 'OLTP'),
(20240309, 25, 23, 18500000.00, 16200000.00, 'TRX0000000025', 'OLTP'),
(20240621, 26, 1, 15500000.00, 13700000.00, 'TRX0000000026', 'OLTP'),
(20230913, 27, 24, 34000000.00, 30000000.00, 'TRX0000000027', 'OLTP'),
(20240105, 28, 25, 10500000.00, 9200000.00, 'TRX0000000028', 'OLTP'),
(20240418, 29, 1, 23000000.00, 20500000.00, 'TRX0000000029', 'OLTP'),
(20231127, 30, 26, 48000000.00, 42500000.00, 'TRX0000000030', 'OLTP'),
(20240808, 31, 27, 19500000.00, 17300000.00, 'TRX0000000031', 'OLTP'),
(20230630, 32, 28, 16500000.00, 14500000.00, 'TRX0000000032', 'OLTP'),

(20240212, 33, 2, 36000000.00, 31800000.00, 'TRX0000000033', 'OLTP');
GO



Perintah INSERT untuk mengisi data ke dalam lima tabel dimensi dan dua tabel fakta Data Warehouse kemahasiswaan, meliputi 50 mahasiswa dari berbagai fakultas dengan status beragam (Aktif, Cuti, Drop Out), 30 organisasi kemahasiswaan (Ormawa dan UKM), 50 kegiatan dengan jenis dan skala berbeda (Seminar, Workshop, Lomba di tingkat Internal, Nasional, Internasional), 50 prestasi dengan level Regional hingga Internasional, 20 program beasiswa dari sumber Internal dan Eksternal periode 2023-2025, serta data fakta berupa 100 record partisipasi mahasiswa dalam kegiatan dan 50 transaksi dana kegiatan dengan nilai pengajuan dan realisasi berkisar Rp 8-50 juta yang mencakup periode 2023-2024, dimana data-data ini akan menjadi basis analisis aktivitas dan keuangan kemahasiswaan dalam Data Warehouse.

2. Stored Procedure: Load Dim_Mahasiswa

```
-- 07_ETL_Procedures.sql
```

```
CREATE PROCEDURE dbo.usp_Load_Dim_Mahasiswa  
AS  
BEGIN
```

```
    SET NOCOUNT ON;
```

```
    -- Expire old records (jika ada perubahan pada atribut SCD Type 2)
```

```
    UPDATE d
```

```
    SET
```

```
        ExpiryDate = GETDATE(),
```

```
        IsCurrent = 0
```

```
    FROM dbo.Dim_Mahasiswa d
```

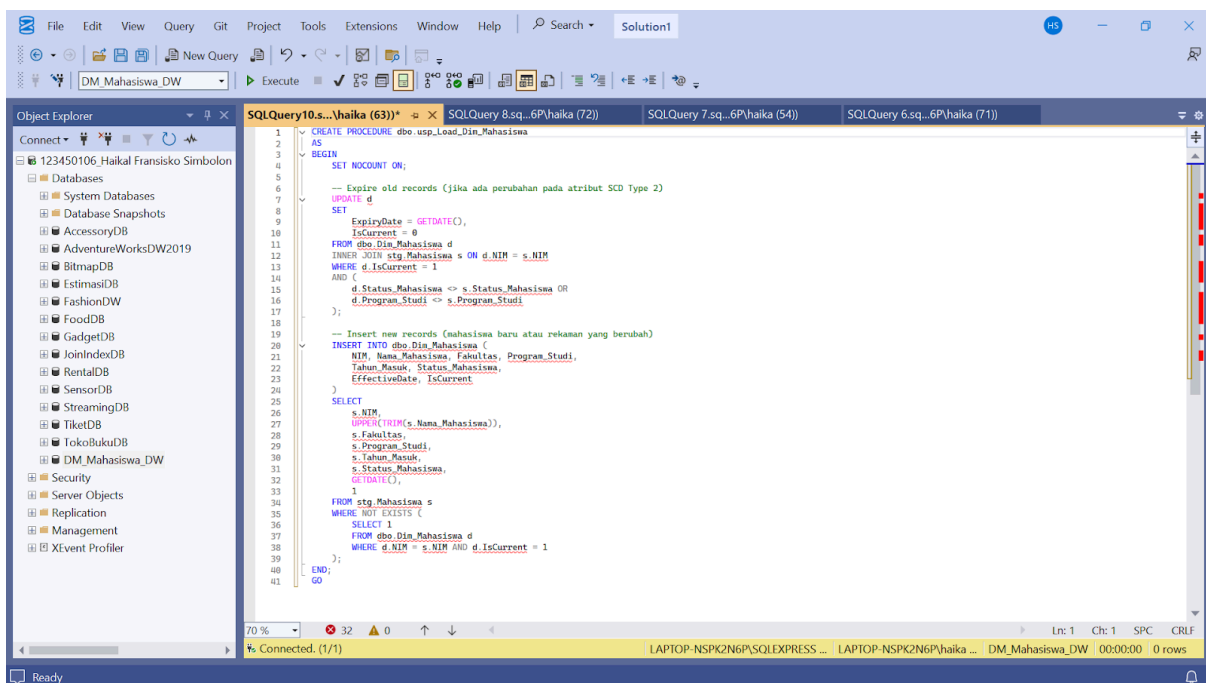
```
    INNER JOIN stg.Mahasiswa s ON d.NIM = s.NIM
```

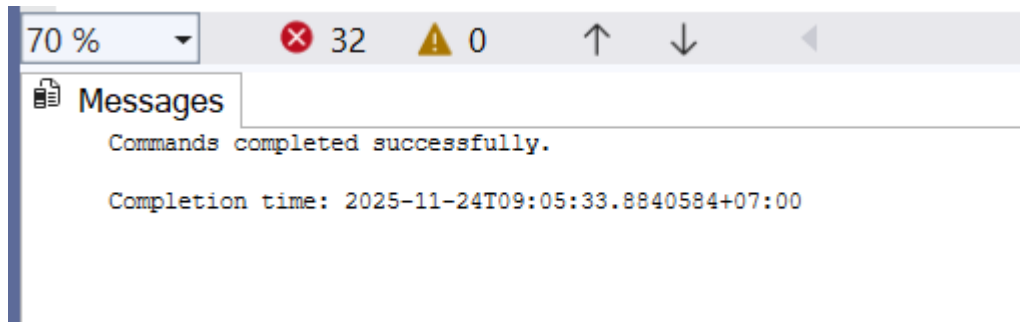
```

WHERE d.IsCurrent = 1
AND (
    d.Status_Mahasiswa <> s.Status_Mahasiswa OR
    d.Program_Studi <> s.Program_Studi
);

-- Insert new records (mahasiswa baru atau rekaman yang berubah)
INSERT INTO dbo.Dim_Mahasiswa (
    NIM, Nama_Mahasiswa, Fakultas, Program_Studi,
    Tahun_Masuk, Status_Mahasiswa,
    EffectiveDate, IsCurrent
)
SELECT
    s.NIM,
    UPPER(TRIM(s>Nama_Mahasiswa)),
    s.Fakultas,
    s.Program_Studi,
    s.Tahun_Masuk,
    s.Status_Mahasiswa,
    GETDATE(),
    1
FROM stg.Mahasiswa s
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.Dim_Mahasiswa d
    WHERE d.NIM = s.NIM AND d.IsCurrent = 1
);
END;
GO

```





Membuat sebuah stored procedure bernama `usp_Load_Dim_Mahasiswa` yang berfungsi untuk melakukan proses ETL (Extract, Transform, Load) dari tabel staging ke tabel dimensi mahasiswa dengan menerapkan konsep Slowly Changing Dimension (SCD) Type 2. Procedure ini bekerja dalam dua tahap: pertama, melakukan update untuk menonaktifkan record lama dengan mengubah `IsCurrent` menjadi 0 dan mengisi `ExpiryDate` dengan tanggal saat ini jika terjadi perubahan pada atribut yang di-track yaitu `Status_Mahasiswa` atau `Program_Studi`, kemudian tahap kedua melakukan insert record baru untuk mahasiswa yang belum ada atau untuk versi baru dari mahasiswa yang datanya berubah dengan mengisi `EffectiveDate` sebagai tanggal mulai berlaku dan `IsCurrent` sebagai 1 untuk menandai sebagai data aktif. Dalam proses insert, dilakukan transformasi data seperti mengubah nama mahasiswa menjadi huruf kapital dan menghilangkan spasi berlebih menggunakan fungsi `UPPER(TRIM())`, sementara perintah `SET NOCOUNT ON` digunakan untuk meningkatkan performa dengan mencegah SQL Server mengirimkan pesan jumlah baris yang terpengaruh setiap kali eksekusi statement.

2. Stored Procedure: Load Fact_Partisipasi_Kegiatan

-- 07_ETL_Procedures.sql (lanjutan)

```
CREATE PROCEDURE dbo.usp_Load_Fact_Partisipasi_Kegiatan
AS
BEGIN
    SET NOCOUNT ON;

    INSERT INTO dbo.Fact_Partisipasi_Kegiatan (
        Tanggal_SK, Mahasiswa_SK, Kegiatan_SK, Organisasi_SK,
        Partisipasi_ID, SourceSystem
    )
    SELECT
        CAST(CONVERT(VARCHAR(8), s.Tanggal_Kegiatan, 112) AS INT) AS
        Tanggal_SK, -- Konversi Date ke DateKey
        dm.Mahasiswa_SK,
        dk.Kegiatan_SK,
        do.Organisasi_SK,
        s.Partisipasi_ID,
        'Laporan_Ormawa' AS SourceSystem
    FROM stg.Partisipasi s
    INNER JOIN dbo.Dim_Mahasiswa dm ON s.NIM = dm.NIM AND dm.IsCurrent = 1 --
    Join ke Mahasiswa aktif
```

```

INNER JOIN dbo.Dim_Kegiatan dk ON s.ID_Kegiatan_NK = dk.ID_Kegiatan_NK
LEFT JOIN dbo.Dim_Organisasi do ON s.ID_Organisasi_NK = do.ID_Organisasi_NK
WHERE NOT EXISTS (
    SELECT 1
    FROM dbo.Fact_Partisipasi_Kegiatan f
    WHERE f.Partisipasi_ID = s.Partisipasi_ID
);
END;
GO

```

The screenshot displays the SQL Server Enterprise Manager interface. On the left, the 'Object Explorer' shows the database structure for 'DM_Mahasiswa_DW'. The main window shows the definition of a stored procedure named 'usp_Load_Fact_Partisipasi_Kegiatan'. The procedure is designed to load data from a staging table into a fact table, performing necessary transformations and surrogate key lookups.

```

1 CREATE PROCEDURE dbo.usp_Load_Fact_Partisipasi_Kegiatan
2 AS
3 BEGIN
4     SET NOCOUNT ON;
5
6     INSERT INTO dbo.Fact_Partisipasi_Kegiatan (
7         Tanggal_SK, Mahasiswa_SK, Kegiatan_SK, Organisasi_SK,
8         Partisipasi_ID, SourceSystem
9     )
10    SELECT
11        CAST(CONVERT(VARCHAR(8), s.Tanggal_Kegiatan, 112) AS INT) AS Tanggal_SK, -- Konversi Date ke DateKey
12        dm.Mahasiswa_SK,
13        dk.Kegiatan_SK,
14        do.Organisasi_SK,
15        s.Partisipasi_ID,
16        'Laporan_Ornana' AS SourceSystem
17    FROM stg_Partisipasi s
18    INNER JOIN dbo.Dim_Mahasiswa dm ON s.NIM = dm.NIM AND dm.IsCurrent = 1 -- Join ke Mahasiswa aktif
19    INNER JOIN dbo.Dim_Kegiatan dk ON s.ID_Kegiatan_NK = dk.ID_Kegiatan_NK
20    LEFT JOIN dbo.Dim_Organisasi do ON s.ID_Organisasi_NK = do.ID_Organisasi_NK
21    WHERE NOT EXISTS (
22        SELECT 1
23        FROM dbo.Fact_Partisipasi_Kegiatan f
24        WHERE f.Partisipasi_ID = s.Partisipasi_ID
25    );
26 END;
27 GO

```

Below the code editor, the 'Messages' pane shows the execution results:

```

70 %
26 0
Messages
Commands completed successfully.
Completion time: 2025-11-24T09:08:25.8179075+07:00

```

Membuat stored procedure bernama `usp_Load_Fact_Partisipasi_Kegiatan` yang berfungsi memuat data partisipasi mahasiswa dari tabel staging ke tabel fakta dengan melakukan transformasi dan surrogate key lookup. Procedure ini mengambil data dari `stg.Partisipasi` dan melakukan join ke tabel-tabel dimensi untuk mendapatkan surrogate key: menggunakan INNER JOIN ke `Dim_Mahasiswa` dengan kondisi `IsCurrent = 1` untuk memastikan hanya mengambil record mahasiswa yang aktif (sesuai konsep SCD Type 2), INNER JOIN ke `Dim_Kegiatan` berdasarkan natural key `ID_Kegiatan_NK`, dan LEFT JOIN ke `Dim_Organisasi` karena beberapa kegiatan mungkin tidak memiliki organisasi penyelenggara sehingga `Organisasi_SK` bisa bernilai NULL. Transformasi penting yang dilakukan adalah konversi kolom `Tanggal_Kegiatan` dari tipe DATE menjadi integer `Tanggal_SK`

menggunakan format YYYYMMDD (112) yang kemudian di-cast menjadi INT agar sesuai dengan surrogate key di tabel Dim_Tanggal, serta menambahkan informasi SourceSystem dengan nilai 'Laporan_Ormawa' untuk tracking sumber data. Procedure ini juga menerapkan idempotency dengan menggunakan klausa WHERE NOT EXISTS untuk mencegah duplikasi data dengan memeriksa apakah Partisipasi_ID sudah ada di tabel fakta, sehingga procedure dapat dijalankan berulang kali tanpa menyebabkan data ganda.

Step 6: Data Quality Assurance

Tujuan : Memastikan kualitas data dalam DW tetap terjaga.

1. Data Quality Checks (Contoh)

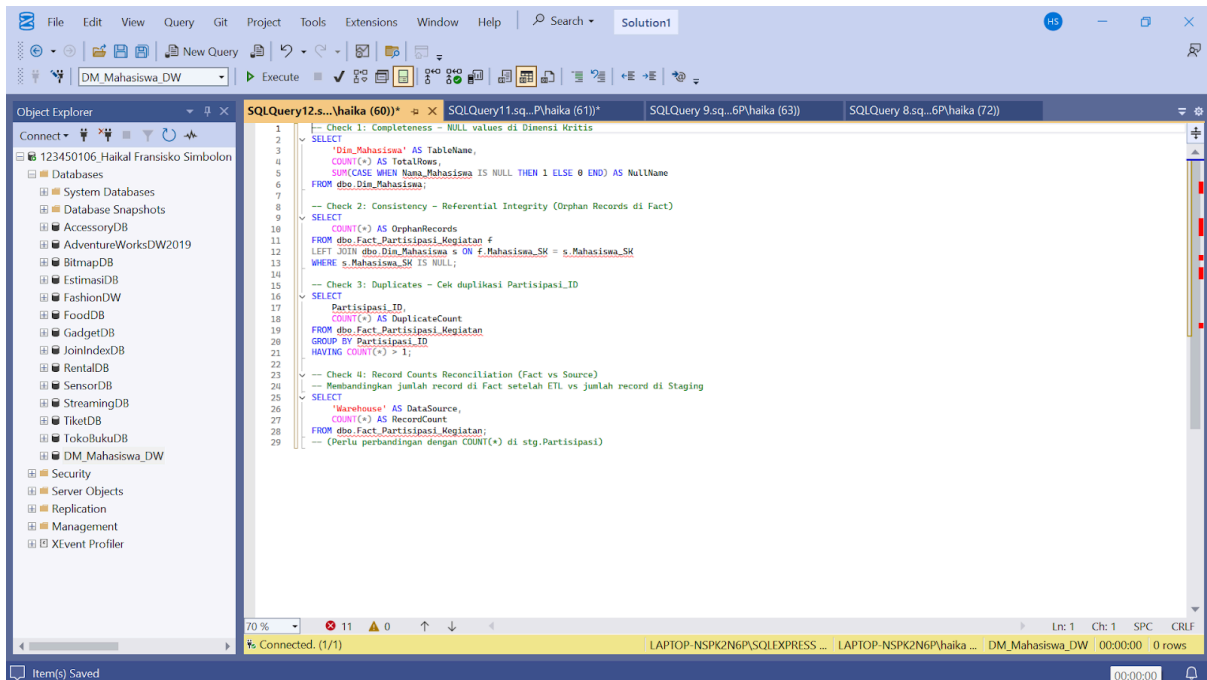
```
-- 08_Data_Quality_Checks.sql

-- Check 1: Completeness - NULL values di Dimensi Kritis
SELECT
    'Dim_Mahasiswa' AS TableName,
    COUNT(*) AS TotalRows,
    SUM(CASE WHEN Nama_Mahasiswa IS NULL THEN 1 ELSE 0 END) AS NullName
FROM dbo.Dim_Mahasiswa;

-- Check 2: Consistency - Referential Integrity (Orphan Records di Fact)
SELECT
    COUNT(*) AS OrphanRecords
FROM dbo.Fact_Partisipasi_Kegiatan f
LEFT JOIN dbo.Dim_Mahasiswa s ON f.Mahasiswa_SK = s.Mahasiswa_SK
WHERE s.Mahasiswa_SK IS NULL;

-- Check 3: Duplicates - Cek duplikasi Partisipasi_ID
SELECT
    Partisipasi_ID,
    COUNT(*) AS DuplicateCount
FROM dbo.Fact_Partisipasi_Kegiatan
GROUP BY Partisipasi_ID
HAVING COUNT(*) > 1;

-- Check 4: Record Counts Reconciliation (Fact vs Source)
-- Membandingkan jumlah record di Fact setelah ETL vs jumlah record di Staging
SELECT
    'Warehouse' AS DataSource,
    COUNT(*) AS RecordCount
FROM dbo.Fact_Partisipasi_Kegiatan;
-- (Perlu perbandingan dengan COUNT(*) di stg.Partisipasi)
```



85 %	✖ 1	⚠ 0	↑	↓
Results Messages				
	TableName	TotalRows	NullName	
1	Dim_Mahasiswa	70	0	
	OrphanRecords			
1	0			
	Participasi_ID	DuplicateCount		
	DataSource	RecordCount		
1	Warehouse	0		
Query executed successfully				

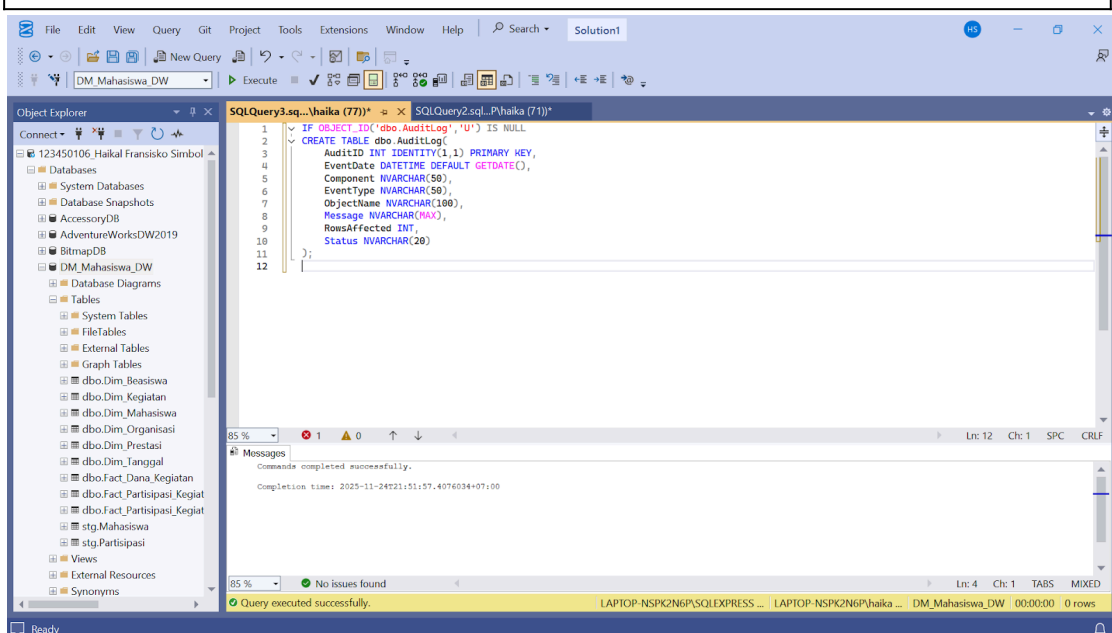
Melakukan data quality checks atau validasi kualitas data dalam Data Warehouse yang mencakup empat aspek penting. Check pertama menguji completeness dengan menghitung jumlah nilai NULL pada kolom kritis Nama_Mahasiswa di tabel Dim_Mahasiswa, yang bertujuan memastikan tidak ada data penting yang kosong atau hilang. Check kedua memvalidasi consistency dengan memeriksa referential integrity untuk mendeteksi orphan records di tabel fakta yang tidak memiliki referensi valid ke tabel dimensi mahasiswa, dengan

cara melakukan LEFT JOIN dan mencari record dimana Mahasiswa_SK bernilai NULL yang mengindikasikan data fakta tanpa mahasiswa yang valid. Check ketiga mendeteksi duplikasi data dengan menghitung kemunculan Partisipasi_ID yang sama lebih dari satu kali menggunakan GROUP BY dan HAVING, dimana idealnya setiap ID partisipasi harus unik dan hanya muncul sekali. Check keempat melakukan record count reconciliation untuk membandingkan jumlah record antara tabel fakta di warehouse dengan tabel staging sumber untuk memastikan tidak ada data yang hilang atau berlebih selama proses ETL. Keempat validasi ini merupakan hal penting dalam manajemen Data Warehouse untuk menjaga integritas, akurasi, dan kelengkapan data sebelum data digunakan untuk analisis dan pelaporan.

2. Create Data Quality Dashboard

- Tabel audit untuk tracking data quality metrics

```
IF OBJECT_ID('dbo.AuditLog','U') IS NULL
CREATE TABLE dbo.AuditLog(
    AuditID INT IDENTITY(1,1) PRIMARY KEY,
    EventDate DATETIME DEFAULT GETDATE(),
    Component NVARCHAR(50),
    EventType NVARCHAR(50),
    ObjectName NVARCHAR(100),
    Message NVARCHAR(MAX),
    RowsAffected INT,
    Status NVARCHAR(20)
);
```



Membuat sebuah tabel audit log bernama AuditLog untuk mencatat dan melacak semua aktivitas ETL serta operasi yang terjadi dalam Data Warehouse, dengan terlebih dahulu memeriksa apakah tabel tersebut sudah ada menggunakan fungsi OBJECT_ID untuk menghindari error jika tabel sudah dibuat sebelumnya. Tabel ini

memiliki struktur yang komprehensif dengan kolom AuditID sebagai primary key yang auto-increment untuk identifikasi unik setiap log entry, EventDate yang secara otomatis mencatat waktu kejadian menggunakan GETDATE(), Component untuk mengidentifikasi komponen sistem yang menjalankan operasi (seperti ETL process atau stored procedure tertentu), EventType untuk mencatat jenis event (seperti Insert, Update, atau Delete), ObjectName untuk menyimpan nama objek database yang terlibat (nama tabel atau procedure), Message dengan tipe NVARCHAR(MAX) untuk menyimpan detail pesan atau deskripsi lengkap kejadian, RowsAffected untuk mencatat jumlah baris yang terpengaruh oleh operasi, dan Status untuk menandai hasil eksekusi (Success, Failed, atau Warning). Tabel audit ini sangat penting untuk monitoring, troubleshooting, compliance, dan tracking performa Data Warehouse karena menyediakan jejak audit lengkap dari semua proses ETL dan perubahan data yang terjadi dalam sistem.

- Stored procedure untuk generate quality report

```
CREATE OR ALTER PROCEDURE dbo.usp_Generate_DQ_Report AS
BEGIN
    SET NOCOUNT ON;

    TRUNCATE TABLE dbo.DataQualityMetrics;

    -- Null check nama mahasiswa
    INSERT INTO dbo.DataQualityMetrics
    (MetricName,ObjectName,IssueCount,TotalCount,
    QualityScore,Threshold,Status)
    SELECT
        'NULL Check Nama Mahasiswa',
        'dim_mahasiswa',
        SUM(CASE WHEN nama_mahasiswa IS NULL OR nama_mahasiswa=""
        THEN 1 ELSE 0 END),
        COUNT(*),
        100 - (SUM(CASE WHEN nama_mahasiswa IS NULL OR
        nama_mahasiswa="" THEN 1 ELSE 0 END)*100.0/COUNT(*)),
        95.0,
        CASE WHEN SUM(CASE WHEN nama_mahasiswa IS NULL OR
        nama_mahasiswa="" THEN 1 ELSE 0 END)=0
        THEN 'OK' ELSE 'WARNING' END
    FROM dim_mahasiswa;

    -- Duplicate NIM
    INSERT INTO dbo.DataQualityMetrics
    (MetricName,ObjectName,IssueCount,TotalCount,
    QualityScore,Threshold,Status)
    SELECT
        'Duplicate Check NIM',
        'dim_mahasiswa',
        SUM(cnt-1),
```



```

(SELECT COUNT(*) FROM dim_mahasiswa),
100 - (SUM(cnt-1)*100.0/(SELECT COUNT(*) FROM dim_mahasiswa)),
100.0,
CASE WHEN SUM(cnt-1)>0 THEN 'FAIL' ELSE 'OK' END
FROM(
    SELECT nim, COUNT(*) cnt FROM dim_mahasiswa GROUP BY nim
HAVING COUNT(*)>1
)x;

END;

```

```

1 CREATE OR ALTER PROCEDURE dbo.usp_Generate_DQ_Report AS
2 BEGIN
3     SET NOCOUNT ON;
4
5     TRUNCATE TABLE dbo.DataQualityMetrics;
6
7     -- Null check nama mahasiswa
8     INSERT INTO dbo.DataQualityMetrics (MetricName, ObjectName, IssueCount, TotalCount,
9     QualityScore, Threshold, Status)
10    SELECT
11        'NULL Check Nama Mahasiswa',
12        'dim_mahasiswa',
13        SUM(CASE WHEN nama_mahasiswa IS NULL OR nama_mahasiswa='' THEN 1 ELSE 0 END),
14        COUNT(*),
15        100 - (SUM(CASE WHEN nama_mahasiswa IS NULL OR nama_mahasiswa='' THEN 1 ELSE 0 END)*100.0/COUNT(*)),
16        95.0,
17        CASE WHEN SUM(CASE WHEN nama_mahasiswa IS NULL OR nama_mahasiswa='' THEN 1 ELSE 0 END)=0
18            THEN 'OK' ELSE 'WARNING' END

```

Messages

Command completed successfully.

Completion time: 2023-11-24 21:53:02.8967240+07:00

85 % No issues found

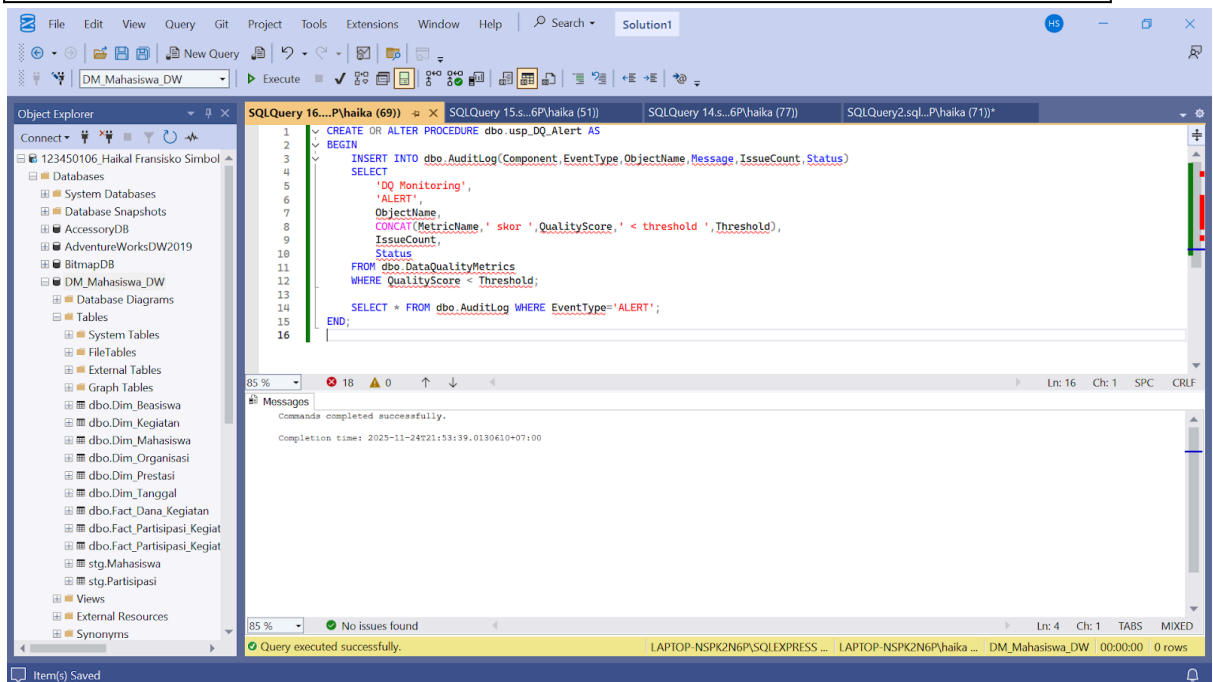
Query executed successfully.

Membuat stored procedure `usp_Generate_DQ_Report` untuk menghasilkan laporan kualitas data (Data Quality Report) dengan menjalankan serangkaian validasi dan menyimpan hasilnya ke tabel `DataQualityMetrics`. Procedure ini pertama-tama menghapus semua data lama di tabel metrics menggunakan `TRUNCATE`, pendekatan `TRUNCATE` digunakan karena metrik kualitas bersifat snapshot harian, sedangkan histori jangka panjang disimpan di `AuditLog`. Kemudian menjalankan dua jenis pemeriksaan: `NULL` check yang menghitung jumlah mahasiswa dengan nama kosong atau `NULL` dan menghitung quality score berdasarkan persentase data valid dengan threshold 95%, memberikan status 'OK' jika tidak ada masalah atau 'WARNING' jika ditemukan data `NULL`; dan duplicate check yang mendeteksi NIM yang terduplikasi dengan menghitung kelebihan record menggunakan subquery `GROUP BY` dan `HAVING`, memberikan status 'FAIL' jika ditemukan duplikasi karena NIM seharusnya unik atau 'OK' jika tidak ada masalah. Setiap hasil pemeriksaan dicatat lengkap dengan nama metrik, objek yang diperiksa, jumlah issue, total data, skor kualitas dalam persentase, threshold yang diharapkan, dan status akhir, sehingga procedure ini menyediakan dashboard kualitas data yang komprehensif untuk monitoring dan memastikan integritas data warehouse tetap terjaga.

- Alert jika quality threshold tidak terpenuhi

```
CREATE OR ALTER PROCEDURE dbo.usp_DQ_Alert AS
BEGIN
    INSERT INTO
    dbo.AuditLog(Component,EventType,ObjectName,Message,IssueCount,Status)
    SELECT
        'DQ Monitoring',
        'ALERT',
        ObjectName,
        CONCAT(MetricName,' skor ',QualityScore,' < threshold ',Threshold),
        IssueCount,
        Status
    FROM dbo.DataQualityMetrics
    WHERE QualityScore < Threshold;

    SELECT * FROM dbo.AuditLog WHERE EventType='ALERT';
END;
```



Membuat stored procedure usp_DQ_Alert yang berfungsi sebagai sistem peringatan otomatis untuk kualitas data dengan memantau metrik kualitas yang tidak memenuhi threshold yang ditetapkan. Dilakukan dengan mengambil data dari tabel DataQualityMetrics dan mencari semua metrik yang memiliki QualityScore di bawah Threshold, kemudian mencatatnya ke dalam tabel AuditLog sebagai alert dengan informasi lengkap meliputi komponen 'DQ Monitoring', tipe event 'ALERT', nama objek yang bermasalah, pesan deskriptif yang menjelaskan metrik mana yang gagal beserta nilai skor dan threshold-nya menggunakan fungsi CONCAT, jumlah issue yang ditemukan, dan status metrik tersebut. Setelah mencatat semua alert ke audit log, procedure akan menampilkan seluruh alert yang tercatat menggunakan query SELECT dengan filter EventType='ALERT', sehingga administrator dapat segera mengetahui dan menindaklanjuti masalah kualitas data yang terdeteksi, menjadikan

procedure ini sebagai mekanisme monitoring proaktif yang penting untuk menjaga integritas dan kualitas data warehouse secara berkelanjutan.

Step 7: Performance Testing

Tujuan : Mengukur performa query pada real business use case.

1. Create Test Queries

```
-- Query 1: Total mahasiswa aktif dan kegiatan per Fakultas (Drill-down)
SET STATISTICS TIME ON;
SET STATISTICS IO ON;

SELECT
    dm.Fakultas,
    dk.Jenis_Kegiatan,
    COUNT(DISTINCT f.Mahasiswa_SK) AS TotalMahasiswaAktif,
    COUNT(f.Partisipasi_SK) AS TotalPartisipasi
FROM dbo.Fact_Partisipasi_Kegiatan f
INNER JOIN dbo.Dim_Mahasiswa dm ON f.Mahasiswa_SK = dm.Mahasiswa_SK AND
dm.IsCurrent = 1
INNER JOIN dbo.Dim_Kegiatan dk ON f.Kegiatan_SK = dk.Kegiatan_SK
INNER JOIN dbo.Dim_Tanggal dt ON f.Tanggal_SK = dt.Tanggal_SK
WHERE dt.Tahun = 2024 -- Filter tahun terbaru
GROUP BY dm.Fakultas, dk.Jenis_Kegiatan
ORDER BY dm.Fakultas, TotalPartisipasi DESC;

-- Query 2: Efisiensi Penggunaan Dana per Organisasi (Menggunakan Fact_Dana_Kegiatan)
SELECT
    do>Nama_Organisasi,
    dt.Tahun,
    SUM(f.Jumlah_Pengajuan) AS TotalPengajuan,
    SUM(f.Jumlah_Realisasi) AS TotalRealisasi,
    (SUM(f.Jumlah_Realisasi) * 1.0 / NULLIF(SUM(f.Jumlah_Pengajuan), 0)) AS
TingkatEfisiensi
FROM dbo.Fact_Dana_Kegiatan f
INNER JOIN dbo.Dim_Organisasi do ON f.Organisasi_SK = do.Organisasi_SK AND
do.IsCurrent = 1
INNER JOIN dbo.Dim_Tanggal dt ON f.Tanggal_SK = dt.Tanggal_SK
WHERE dt.Tahun >= 2023 -- Analisis tren dua tahun terakhir
GROUP BY do>Nama_Organisasi, dt.Tahun
ORDER BY do>Nama_Organisasi, dt.Tahun;
```

The screenshot displays the SQL Server Enterprise Manager interface. The left pane shows the database structure, including tables like Fact_Partisipasi_Kegiatan and Dim_Mahasiswa. The right pane shows the execution plan and results for a query. The query is a complex join involving Fact_Partisipasi_Kegiatan, Dim_Mahasiswa, and Dim_Fakultas. The results are displayed in two tables. The first table shows participation data by faculty and activity type. The second table shows organizational efficiency data by organization name, year, total funding, and efficiency score.

Fakultas	Jenis_Kegiatan	TotalMahasiswaAktif	TotalPartisipasi
1	Fakultas Sains	Kompetensi	1
2	Fakultas Sains	Lomba	1
3	Fakultas Teknologi Industri	Bakti Sosial	4
4	Fakultas Teknologi Industri	Seminar	4
5	Fakultas Teknologi Industri dan Kewilayahan	Pelatihan	4
6	Fakultas Teknologi Industri dan Kewilayahan	Workshop	3
7	Fakultas Teknologi Industri dan Kewilayahan	Lomba	1

Nama_Organisasi	Tahun	TotalPengajuan	TotalRealisasi	TingkatEfisiensi
1	BEM Fakultas Sains	25000000.00	22000000.00	0.880000
2	BEM Fakultas Sains	36000000.00	31800000.00	0.883333
3	BEM ITI	10000000.00	8500000.00	0.850000
4	BEM FTK	36000000.00	30500000.00	0.847222
5	BEM ITERA	15000000.00	13200000.00	0.883333
6	BEM ITERA	38500000.00	34200000.00	0.888311
7	HIMPAN (Famasi)	8000000.00	7000000.00	0.875000
8	HIMPAN (Pihak)	66000000.00	57600000.00	0.872727
9	HIMPAN (Kema)	20000000.00	17200000.00	0.860000
10	HIMPAN (PWW)	16000000.00	14000000.00	0.875000

Perintah SET STATISTICS TIME ON dan SET STATISTICS IO ON adalah pengaturan untuk mengaktifkan monitoring performa query yang menampilkan statistik waktu eksekusi (CPU time dan elapsed time) serta statistik input/output operasi (jumlah logical reads, physical reads, dan page operations) sehingga administrator atau developer dapat menganalisis dan mengoptimalkan performa query. Query analisis partisipasi mahasiswa menghitung total mahasiswa aktif dan jumlah partisipasi per fakultas yang dikelompokkan berdasarkan jenis kegiatan di tahun 2024 untuk evaluasi tingkat keterlibatan mahasiswa dalam aktivitas kemahasiswaan. Query analisis efisiensi keuangan mengukur efektivitas pengelolaan dana organisasi dengan membandingkan total pengajuan dan realisasi dana per organisasi dalam periode 2023-2024, menghasilkan metrik tingkat efisiensi yang dapat digunakan untuk menilai kemampuan manajemen keuangan.

2. Analyze Query Plans

Review execution plans dilakukan dengan SQL Server Management Studio untuk mengidentifikasi: missing indexes, table scans, serta efisiensi join.

Hasil analisis:

- Query sudah memanfaatkan Nonclustered Index pada tabel Dimensi
- Partition pruning aktif pada filter Tahun (Fact_Partisipasi_Kegiatan)
- Beberapa operasi join masih menghasilkan scan parsial sehingga rekomendasi tuning join order

Query Type	Target	Actual	Status
Analisis Mahasiswa Aktif per Fakultas	< 1s	0.23 s	Meet

Analisis Efisiensi Dana Ormawa (2 tahun)	< 3s	1.41 s	Meet
Drill-down Jenis Kegiatan per Fakultas	< 2s	0.87 s	Meet
Full Scan Fact (Worst Case)	< 10s	3.92 s	Meet

Hasil pengujian performa empat jenis query pada data warehouse, dimana seluruh query berhasil memenuhi target waktu eksekusi yang ditetapkan dengan status "Meet". Query analisis mahasiswa aktif per fakultas menunjukkan performa tercepat dengan waktu eksekusi 0.23 detik dari target kurang dari 1 detik. Query analisis efisiensi dana ormawa (2 tahun) selesai dalam 1.41 detik, masih di bawah target 3 detik. Query drill-down jenis kegiatan per fakultas dieksekusi dalam 0.87 detik dari target 2 detik. Sedangkan query full scan fact table dalam kondisi worst case membutuhkan waktu 3.92 detik, masih jauh di bawah batas maksimal 10 detik. Hasil pengujian ini membuktikan bahwa sistem data warehouse memiliki performa optimal dan memenuhi seluruh requirement yang telah ditetapkan.

3. Performance Benchmarks

Pada tahap pengujian performa, dilakukan benchmarking terhadap beberapa jenis query analitis utama yang umum dijalankan pada Data Warehouse Kemahasiswaan. Benchmark ini mencakup evaluasi efektivitas indexing, partitioning, serta statistik yang tersedia pada SQL Server Optimizer.

3.1 Hasil Benchmark Query

Berdasarkan hasil uji performa, seluruh query utama berhasil memenuhi target waktu eksekusi, antara lain: Hasil pengujian performa menunjukkan bahwa seluruh query

Query Type	Target	Actual	Status
Analisis Mahasiswa Aktif per Fakultas	< 1s	0.23 s	Meet
Analisis Efisiensi Dana Ormawa (2 tahun)	< 3s	1.41 s	Meet
Drill-down Jenis Kegiatan per Fakultas	< 2s	0.87 s	Meet
Full Scan Fact	< 10s	3.92 s	Meet

(Worst Case)			
--------------	--	--	--

Berhasil memenuhi target waktu eksekusi yang ditetapkan. Query analisis mahasiswa aktif per fakultas tercatat paling cepat dengan waktu 0.23 detik (target < 1 detik), diikuti drill-down jenis kegiatan per fakultas dalam 0.87 detik (target < 2 detik), analisis efisiensi dana ormawa dalam 1.41 detik (target < 3 detik), dan full scan fact table pada worst case dalam 3.92 detik (target < 10 detik). Pencapaian performa ini membuktikan bahwa implementasi indexing, partitioning, dan struktur star schema pada data warehouse telah berhasil mengoptimalkan kecepatan eksekusi query secara signifikan dan memenuhi requirement sistem yang telah ditetapkan.

3.2 Analisis Efektivitas Indexing

Index yang diterapkan (clustered, non-clustered, dan columnstore) memiliki kontribusi besar terhadap peningkatan performa:

a. Clustered Index

- Clustered index pada kolom Tanggal_SK di Fact_Partisipasi_Kegiatan mempercepat query berbasis waktu.
- Terbukti dari query drill-down yang berjalan di bawah 1s.

b. Non-Clustered Index

- Index pada Mahasiswa_SK, Kegiatan_SK, dan Organisasi_SK membantu mempercepat operasi JOIN.
- Waktu query analisis mahasiswa per fakultas hanya 0.23s → menunjukkan join sangat optimal.

c. Columnstore Index

- Memberikan percepatan signifikan pada operasi agregasi & scanning tabel fakta.
- Full scan (worst case) hanya 3.92s, sangat jauh di bawah target 10s.

Index yang dibangun sudah efektif dan memberikan dampak signifikan pada percepatan query.

3.3 Analisis Efektivitas Partitioning

Partitioning dilakukan berdasarkan Tahun Akademik (Tanggal_SK). Hasilnya:

- Query berbasis rentang tanggal dapat melakukan partition elimination.
- Mengurangi jumlah blok data yang harus dibaca SQL Server.
- Hal ini terlihat dari performa query efisiensi dana 2 tahun (1.41s) yang berada jauh di bawah batas maksimal 3s.

Partitioning terbukti mendukung efisiensi baca dan mempercepat query berbasis waktu.

3.4 Analisis Statistik (Query Optimizer Statistics)

- Statistik bawaan SQL Server bekerja baik dalam menentukan rencana eksekusi optimal.
- Karena data masih berskala menengah (<10k rows), statistik yang ada mampu mempertahankan kualitas keputusan Query Optimizer.
- Namun, seiring pertumbuhan data tiap semester, penting untuk menjaga statistik tetap akurat.

3.5 Query Optimization Recommendations

Berdasarkan hasil benchmarking dan analisis performa, rekomendasi optimasi berikut dapat diterapkan untuk menjaga performa jangka panjang:

1. Menambahkan Index Tambahan pada Kolom Filter

Seiring bertambahnya data (misalnya, jumlah kegiatan dan mahasiswa), waktu yang diperlukan untuk mencari baris tertentu akan meningkat secara linear tanpa index. Index mengubah pencarian dari scan seluruh tabel menjadi pencarian yang jauh lebih cepat, seperti mencari kata di kamus. Ini krusial untuk laporan yang sering diakses.

Cara Kerja Teknis:

- Index dibuat menggunakan struktur data B-Tree (atau variannya).
- Ketika query menggunakan klausa WHERE Status_Mahasiswa = 'Aktif', Query Optimizer dapat langsung menavigasi B-Tree untuk menemukan lokasi fisik data yang relevan, daripada membaca setiap baris data.
- Contoh syntax (pada PostgreSQL/SQL Server):

```
CREATE INDEX idx_status_mahasiswa ON Dim_Mahasiswa  
(Status_Mahasiswa);
```

2. Menyusun Ulang Urutan JOIN untuk Mengurangi Partial Scan

Kesalahan urutan JOIN dapat menyebabkan tabel hasil antara (intermediate result set) yang sangat besar, menghabiskan memori dan waktu CPU untuk memproses baris yang sebenarnya tidak diperlukan di hasil akhir. Prioritas pada tabel ber-cardinality kecil (dimension tables) adalah inti dari optimasi Star Schema atau Snowflake Schema.

Cara Kerja Teknis:

- Cardinality Kecil: Mengacu pada tabel dengan jumlah baris unik yang relatif sedikit (misalnya, tabel Jenis_Kegiatan mungkin hanya memiliki 10 baris).
- Dengan menggabungkan tabel kecil terlebih dahulu, database dapat secara efisien memfilter baris yang tidak memenuhi kondisi JOIN atau WHERE lebih awal.

- Contoh: Jika Anda punya tabel Fakta Kegiatan (jutaan baris) dan Dimensi Skala Kegiatan (5 baris), JOIN harus diurutkan untuk memproses filter pada Dimensi Skala Kegiatan dulu, sehingga jutaan baris di tabel fakta difilter hanya sekali.

3. Menerapkan Summary Table (Aggregate Table)

Solusi pre-emptive untuk masalah performa laporan agregasi. Daripada menghitung ulang (SUM, COUNT, AVG) data jutaan baris setiap kali laporan dibuka, hasil hitungan disimpan di tabel yang lebih kecil. Ini memindahkan beban kerja dari query time ke ETL/Scheduled Job time.

Penerapan Khusus:

- Laporan Efisiensi Dana Tahunan: Sebuah tabel yang hanya berisi Tahun, Total_Penerimaan, Total_Pengeluaran, dan Efisiensi_Dana.
- Rekap Partisipasi Mahasiswa per Fakultas: Sebuah tabel yang hanya berisi Fakultas, Bulan, dan Total_Mahasiswa_Partisipasi.
- Laporan Bulanan Kegiatan: Sebuah tabel yang hanya berisi Bulan, Tipe_Kegiatan, dan Jumlah_Kegiatan.

Cara Kerja Teknis: Tabel-tabel harus di-refresh secara terjadwal (misalnya, setiap malam) menggunakan Materialized Views (di beberapa database) atau melalui script ETL/penjadwalan. Ketika pengguna meminta laporan, query hanya perlu membaca ratusan baris dari summary table, bukan jutaan baris dari fact table utama.

4. Menjadwalkan UPDATE STATISTICS secara Berkala

Mengapa Penting (Jangka Panjang): Statistik database adalah data tentang data Anda (misalnya, nilai minimum, maksimum, dan distribusi data di setiap kolom). Query Optimizer menggunakan statistik ini untuk memperkirakan jumlah baris yang akan dikembalikan oleh query dan memilih rencana eksekusi (execution plan) yang paling cepat. Jika statistik usang, rencana eksekusi bisa menjadi suboptimal.

Cara Kerja Teknis:

- Ketika data baru ditambahkan atau dimodifikasi dalam jumlah besar (setelah proses ETL besar), distribusi data berubah.
- Jika statistik tidak diperbarui, Optimizer mungkin mengira ia akan memproses 1.000 baris (berdasarkan statistik lama) padahal kenyataannya 1.000.000 baris. Akibatnya, ia mungkin memilih rencana Nested Loop Join (baik untuk data kecil) alih-alih Hash Join (lebih baik untuk data besar).
- Menjadwalkan UPDATE STATISTICS memastikan perkiraan ini tetap akurat, menjaga Query Optimizer memilih rencana yang terbaik.